equilibrium

QUANTUS

,

Quantus Network: Substrate and Node Review

1.1
2026-05-13

Equilibrium Group Oy
Meritullinkatu 1
Helsinki
Finland

TABLE OF CONTENTS

0. EXECUTIVE SUMMARY

Auditor: Equilibrium Group Oy
Client: Quantus Labs LLC

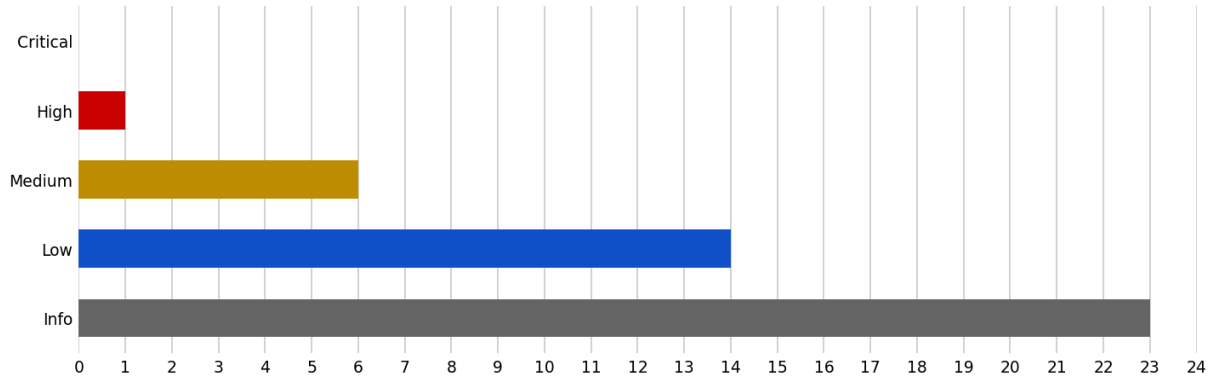
Scope: Technical review of the treasury, multisig, scheduler, and reversible-transfers pallets.

Equilibrium conducted a technical review of the Quantus Network's treasury, multisig, scheduler, and reversible-transfers pallets between 2026-03-16 and 2026-04-15. The review was conducted remotely, with an emphasis on correctness, runtime behaviour, and maintainability.

The review covered treasury configuration, multisig proposal and execution logic, scheduled task semantics, and reversible transfer flows, together with a small number of cross-cutting runtime observations where they materially affect those components.

KEY FINDINGS:

We gathered 44 findings and observations during the review. We did not identify any critical-severity issues. We identified 1 high-severity issue, including a multisig fee-model issue where dispatched `RuntimeCall` execution is not accounted for in the transaction weight model. We also identified 6 medium-severity issues, concentrated around scheduler behaviour and reversible-transfer lifecycle management. Additionally, 14 low-severity findings and 23 informational observations were noted across treasury configuration, scheduler semantics, weight accounting, documentation drift, and missing test coverage.



The most consequential themes are economic correctness and operational predictability rather than direct exploitability. In particular, the multisig pallet's current fee charging model deserves a deeper pass, the scheduler's timestamp semantics should be reviewed carefully before being treated as a stable external contract, and the reversible-transfers high-security lifecycle should

be made more explicit. A recurring theme throughout the review is the value of making the code more obviously correct: easier to reason about, more explicit in its assumptions, and less reliant on subtle runtime behaviour.

The findings are weighted more toward correctness, operational clarity, and maintainability than toward direct exploitability. The main remediation effort should therefore focus on fee accounting, scheduling semantics, lifecycle design, and clear documentation of intentional behaviours.

1. PROJECT SUMMARY

This technical review assesses the treasury, multisig, scheduler, and reversible-transfers pallets of the Quantus Network blockchain. Our assessment produced findings, observations, and recommendations that are detailed in this report.

The Project

The treasury pallet provides the configuration used to direct part of the protocol's reward flow to a designated treasury account.

The multisig pallet implements shared account control for a set of signers, including proposal creation, approval, execution, expiry handling, deposit recovery, and dissolution.

The scheduler pallet is a fork of the upstream Substrate scheduler extended to support both block-based and timestamp-based agendas, together with retry and cancellation behaviour for scheduled tasks.

The reversible-transfers pallet builds on that scheduler machinery to support delayed and recoverable transfers for high-security accounts.

All four are Substrate/FRAME pallets integrated into the Quantus chain runtime. In a few places, the review also comments on surrounding runtime configuration where it materially affects the behaviour of the reviewed pallets.

Context and Scope

The following repositories and commits were in scope for this review:

- Quantus Chain
  - https://github.com/Quantus-Network/chain
  - Commit: `0467a304bf8210b022c3ac63bd9d65f7606ef2e8`
  - Treasury, multisig, scheduler, and reversible-transfers pallets

The following items were out of scope:

- Crate dependencies
- Additional systems, integrations, or resources
- Other pallets and runtime modules not listed above

Disclaimers

This technical review does not guarantee the absence of vulnerabilities or correctness issues. It is a best-effort, time-boxed analysis intended to identify as many technical issues as possible within the agreed scope. Undiscovered issues may still exist. We recommend continued review and testing beyond this engagement.

Remediation

After follow-up discussions, the client began addressing the findings documented in this report. The following remediation has been completed:

- Quantus Chain
  - https://github.com/Quantus-Network/chain/pull/474
  - Commit: `67d61489ff8053b369b5ece3d2bbdbc62d7676b9` (2026-04-08)

For a detailed breakdown of issue resolution status, see Appendix B: Issue Resolution Status.

2. SUBSTRATE RUNTIME

This chapter collects cross-cutting runtime observations that do not fit cleanly into a single pallet section. They mostly concern surrounding runtime configuration and shared assumptions that affect more than one reviewed component.

EQ-QNT-RUNTIME-O-01: Expected and unexpected errors are not separated consistently

Several error paths appear to represent programmer mistakes or broken internal assumptions, but they are currently surfaced as ordinary dispatch errors. This blurs the line between user-triggerable failures and invariant violations.

In practice, the issue is less about any single call site and more about signalling intent. When an error can only occur if surrounding logic is already wrong, presenting it as normal application flow makes the runtime harder to reason about and weakens the diagnostic value of those failures.

Examples
This pattern appears in more than one place; two representative cases are:

pallets/scheduler/src/lib.rs:796-806

```
796 |     let new_time = Self::resolve_time(new_time)?;
797 |
798 |     let lookup = Lookup::<T>::get(id);
799 |     let (when, index) = lookup.ok_or(Error::<T>::NotFound)?;
800 |
801 |     if new_time == when {
802 |       return Err(Error::<T>::RescheduleNoChange.into());
803 |     }
804 |
805 |     let task = Agenda::<T>::try_mutate(when, |agenda| {
806 |       let task = agenda.get_mut(index as usize).ok_or(Error::<T>::NotFound)?;
```

In `do_reschedule_named`, once `Lookup` has already resolved a task name to `(when, index)`, a later `Error::NotFound` from the agenda path no longer describes a normal “task does not exist” case. It indicates that `Lookup` and `Agenda` have fallen out of sync.

pallets/reversible-transfers/src/lib.rs:608-613

```
608 |     fn do_execute_transfer(tx_id: &T::Hash) -> DispatchResultWithPostInfo {
609 |       let pending = PendingTransfers::<T>::get(tx_id).ok_or(Error::<T>::PendingTxNotFound)?;
610 |
611 |       // get from preimages
```

```
612 |       let (call, _) = T::Preimages::realize::<RuntimeCallOf<T>>(&pending.call)
613 |         .map_err(|_| Error::<T>::CallDecodingFailed)?;
```

In `do_execute_transfer`, `CallDecodingFailed` can only occur if the pallet cannot realize a call preimage that it previously stored and then scheduled for its own later execution. That is closer to an invariant failure than an ordinary user-triggerable dispatch error.

Recommendation

Use `frame_support::traits::Defensive`-style checks for errors that indicate an internal invariant violation rather than a user-triggerable condition. These examples are not exhaustive, so apply the same pattern wherever similar invariant violations exist in the codebase. Reserve ordinary dispatch errors for state transitions and inputs that users can realistically trigger.

EQ-QNT-RUNTIME-O-02: Proof-size weight is carried through the system but neither enforced nor priced

The runtime's current weight model meaningfully constrains `ref_time` and block length, but leaves the proof-size dimension effectively uncapped and unpriced. In the present configuration this is a coherent tradeoff for a solochain, especially where proof data is transient rather than persisted in state.

The relevant runtime configuration is explicit:

runtime/src/configs/mod.rs:69-82

```
69 | const NORMAL_DISPATCH_RATIO: Perbill = Perbill::from_percent(75);
70 |
71 | parameter_types! {
72 |   pub const BlockHashCount: BlockNumber = 4096;
73 |   pub const Version: RuntimeVersion = VERSION;
74 |
75 |   /// We allow for 6 seconds of compute with a 12 second average block time.
76 |   pub RuntimeBlockWeights: BlockWeights = BlockWeights::with_sensible_defaults(
77 |     Weight::from_parts(6u64 * WEIGHT_REF_TIME_PER_SECOND, u64::MAX),
78 |     NORMAL_DISPATCH_RATIO,
79 |   );
80 |   // We estimate to download 5MB blocks it takes a 100Mbs link 600ms and 200ms for 1Gbs link
81 |   // To upload, 10Mbs link takes 4.1s and 100Mbs takes 500ms
82 |   pub RuntimeBlockLength: BlockLength = BlockLength::max_with_normal_ratio(5 * 1024 * 1024,
   NORMAL_DISPATCH_RATIO);
```

Block weight is capped at roughly six seconds of `ref_time`, block length is capped at `5 MiB`, and the proof-size component is left at `u64::MAX`.

runtime/src/configs/mod.rs:413-418

```
413 | impl pallet_transaction_payment::Config for Runtime {
414 |   type RuntimeEvent = RuntimeEvent;
415 |   type OnChargeTransaction =
416 |     FungibleAdapter<Balances, pallet_mining_rewards::TransactionFeesCollector<Runtime>>;
417 |   type WeightToFee = IdentityFee<Balance>;
418 |   type LengthToFee = IdentityFee<Balance>;
```

Transaction fees are also identity-mapped from `Weight` and length, with no separate proof-size pricing component.

Even so, the arrangement is worth recording explicitly. Proof size still acts as a proxy for witness size, and leaving it both unenforced and free means that state-heavy transactions rely entirely on `ref_time` benchmarking as the sole guardrail. That may be a coherent choice today, but it becomes more consequential if light clients, proof-heavy bridges, or stateless validation become important.

Recommendation
Document the intended rationale for treating proof size differently from `ref_time`, and revisit the decision if future chain features make large witnesses or storage proofs a more important part of the cost model.

EQ-QNT-RUNTIME-O-03: Several custom pallets do not declare an explicit storage-version baseline

Three of the audited custom pallets store nontrivial runtime state without declaring an explicit `#[pallet::storage_version(...)]` baseline: treasury, multisig, and reversible-transfers. This does not create a current execution bug by itself, but it does leave future storage changes without an in-code version anchor until the team introduces one.

If those pallets' storage layouts remain unchanged until that point, adding explicit versioning later is operationally close to establishing an initial baseline before the first real migration. The main concern here is upgrade hygiene and consistency rather than an immediate defect.

Examples
Treasury declares the pallet without a storage-version attribute:

pallets/treasury/src/lib.rs:23-24

```
23 |   #[pallet::pallet]
24 |   pub struct Pallet<T>(_);
```

Multisig does the same:

pallets/multisig/src/lib.rs:142-143

```
142 |   #[pallet::pallet]
143 |   pub struct Pallet<T>(_);
```

And reversible-transfers also omits an explicit storage-version baseline:

pallets/reversible-transfers/src/lib.rs:123-124

```
123 |   #[pallet::pallet]
124 |   pub struct Pallet<T>(_);
```

Scheduler is different: the custom fork already declares an explicit storage version:

pallets/scheduler/src/lib.rs:191-194

```
191 |   const STORAGE_VERSION: StorageVersion = StorageVersion::new(4);
192 |
193 |   #[pallet::pallet]
194 |   #[pallet::storage_version(STORAGE_VERSION)]
```

That explicit version does not by itself make scheduler's upgrade path clean. The fork retained an upstream `StorageVersion` of `4` even though it changed core scheduler storage semantics to use timestamp-aware agenda keys and task addresses, and the pallet does not define a corresponding local migration hook. In other words, the custom pallets are inconsistent in both directions: treasury, multisig, and reversible-transfers have no explicit baseline, while scheduler has an explicit version that is not clearly tied to this fork's own storage history.

Recommendation

Set explicit `StorageVersion::new(0)` baselines now for treasury, multisig, and reversible-transfers, and gate future storage changes behind explicit version-checked migrations. For scheduler, treat the current deployed `StorageVersion` as the fork's established baseline, and pair it with a fork-specific migration hook and clear upgrade notes that reflect the actual storage history before the next layout change.

3. TREASURY

EQ-QNT-TREASURY-F-01: Treasury account configuration falls back to the all-zero account

The core problem is not just that the pallet has a zero-account default. It is that the storage and API surface read as though the treasury account were optional, while the runtime behaviour quietly collapses missing configuration back into the all-zero account. The interface therefore advertises optionality, but the implementation behaves as though a placeholder treasury account always exists.

The storage and genesis configuration already point in different directions:

pallets/treasury/src/lib.rs:31-60

```
31 |   /// The treasury account that receives mining rewards.
32 |   #[pallet::storage]
33 |   #[pallet::getter(fn treasury_account)]
34 |   pub type TreasuryAccount<T: Config> = StorageValue<_, T::AccountId, OptionQuery>;
```

```
41 |   #[pallet::genesis_config]
42 |   pub struct GenesisConfig<T: Config> {
43 |     pub treasury_account: T::AccountId,
44 |     pub treasury_portion: u8,
45 |   }
46 |
47 |   impl<T: Config> Default for GenesisConfig<T>
48 |   where
49 |     T::AccountId: From<[u8; 32]>,
50 |   {
51 |     fn default() -> Self {
52 |       Self { treasury_account: [0u8; 32].into(), treasury_portion: 50 }
53 |     }
54 |   }
55 |
56 |   #[pallet::genesis_build]
57 |   impl<T: Config> BuildGenesisConfig for GenesisConfig<T> {
58 |     fn build(&self) {
59 |       assert!(self.treasury_portion <= 100, "Treasury portion must be 0-100");
60 |       TreasuryAccount::<T>::put(self.treasury_account.clone());
```

The runtime accessor then collapses the apparent optionality back into an all-zero fallback:

pallets/treasury/src/lib.rs:103-112

```
103 |     pub fn account_id() -> T::AccountId {
104 |       TreasuryAccount::<T>::get().unwrap_or_else(|| {
105 |         T::AccountId::decode(&mut sp_runtime::traits::TrailingZeroInput::zeroes())
```

```
106 |           .unwrap_or_else(|_| {
107 |             // Fallback: zero account
108 |             T::AccountId::decode(&mut &[0u8; 32][..])
109 |               .unwrap_or_else(|_| panic!("Cannot create fallback AccountId"))
110 |           })
111 |       })
112 |     }
```

The pallet's `GenesisConfig::default()` also bakes the same zero-account assumption into a custom `From<[u8; 32]>` bound:

pallets/treasury/src/lib.rs:41-53

```
41 |   #[pallet::genesis_config]
42 |   pub struct GenesisConfig<T: Config> {
43 |     pub treasury_account: T::AccountId,
44 |     pub treasury_portion: u8,
45 |   }
46 |
47 |   impl<T: Config> Default for GenesisConfig<T>
48 |   where
49 |     T::AccountId: From<[u8; 32]>,
50 |   {
51 |     fn default() -> Self {
52 |       Self { treasury_account: [0u8; 32].into(), treasury_portion: 50 }
53 |     }
```

This is a poor fit for either of the two sensible models. If the treasury account is required, the runtime should fail fast during configuration rather than quietly selecting a placeholder address. If the account is meant to be optional, the `Option` should remain visible in the API instead of being collapsed into a silent default. The current runtime's `AccountId32` type means the terminal `panic!` branch is not live today, but the decode-based fallback chain still leaves block-path code coupled to an implicit zero-account default that no caller ever chose explicitly.

Recommendation
Choose one model and encode it directly. If the account is required, use a mandatory genesis value and remove the fallback path. If it is optional, initialise with `None` and propagate the `Option` through the pallet interface. In either model, reject the zero account explicitly instead of encoding it as a silent default.

EQ-QNT-TREASURY-F-02: Missing `TreasuryPortion` storage collapses silently to a valid zero share

`TreasuryPortion` is stored with `ValueQuery`, so an absent key is interpreted as `0` rather than as missing state. Zero is a currently valid configuration value, which means future migrations or test

setups that accidentally clear the key can silently change reward distribution without surfacing an unset-state signal.

The `TreasuryPortion` storage item uses `ValueQuery`, while `TreasuryAccount` uses `OptionQuery`:

pallets/treasury/src/lib.rs:31-39

```
31 |   /// The treasury account that receives mining rewards.
32 |   #[pallet::storage]
33 |   #[pallet::getter(fn treasury_account)]
34 |   pub type TreasuryAccount<T: Config> = StorageValue<_, T::AccountId, OptionQuery>;
35 |
36 |   /// The portion of mining rewards that goes to treasury (0-100).
37 |   #[pallet::storage]
38 |   #[pallet::getter(fn treasury_portion)]
39 |   pub type TreasuryPortion<T: Config> = StorageValue<_, u8, ValueQuery>;
```

The mining-rewards pallet then interprets that byte directly as the treasury share:

pallets/mining-rewards/src/lib.rs:150-153

```
150 |       let treasury_portion = T::Treasury::portion();
151 |       let treasury_reward =
152 |         Permill::from_percent(u32::from(treasury_portion)).mul_floor(total_reward);
153 |       let miner_reward = total_reward.saturating_sub(treasury_reward);
```

The current deployment paths initialise the value correctly, and the root setter deliberately permits `0..=100`. The issue is therefore not that zero is impossible. The issue is that "unset" and "intentionally configured as zero" collapse to the same runtime value.

Recommendation
If unset state must be distinguishable from an intentional zero share, switch to `OptionQuery` and handle `None` explicitly. If zero is meant to remain valid, add upgrade-time presence checks so future migrations cannot silently drop the key.

EQ-QNT-TREASURY-F-03: Shared runtime test genesis does not initialize treasury pallet storage

The shared runtime test helper pre-funds a pallet-derived treasury account, but it does not assimilate the treasury pallet's genesis config. Tests that rely on this helper can therefore run with a funded treasury-looking account while `TreasuryAccount` is still unset and `TreasuryPortion` still reads its default value.

The common test helper funds a `PalletId`-derived account directly:

runtime/tests/common.rs:19-34

```
19 |   pub fn new_test_ext() -> sp_io::TestExternalities {
20 |     let t = frame_system::GenesisConfig::<Runtime>::default().build_storage().unwrap();
21 |
22 |     let mut ext = sp_io::TestExternalities::new(t);
23 |
24 |     // Add balances in the ext
25 |     ext.execute_with(|| {
26 |       Balances::make_free_balance_be(&Self::account_id(1), 1000 * UNIT);
27 |       Balances::make_free_balance_be(&Self::account_id(2), 1000 * UNIT);
28 |       Balances::make_free_balance_be(&Self::account_id(3), 1000 * UNIT);
29 |       Balances::make_free_balance_be(&Self::account_id(4), 1000 * UNIT);
30 |       // Set up treasury account for volume fee collection
31 |       let treasury_pallet_id = PalletId(*b"py/trsry");
32 |       let treasury_account = treasury_pallet_id.into_account_truncating();
33 |       Balances::make_free_balance_be(&treasury_account, 1000 * UNIT);
34 |     });
```

But the treasury pallet's actual genesis storage is only populated when its `GenesisConfig` is built:

pallets/treasury/src/lib.rs:56-62

```
56 |   #[pallet::genesis_build]
57 |   impl<T: Config> BuildGenesisConfig for GenesisConfig<T> {
58 |     fn build(&self) {
59 |       assert!(self.treasury_portion <= 100, "Treasury portion must be 0-100");
60 |       TreasuryAccount::<T>::put(self.treasury_account.clone());
61 |       TreasuryPortion::<T>::put(self.treasury_portion);
62 |     }
```

This is not a production-chain wiring bug. The shipped runtime presets do assimilate treasury genesis correctly. The problem is narrower: shared tests can accidentally exercise fallback treasury behaviour while looking as though they are using the production treasury account.

Recommendation
Either assimilate `pallet_treasury::GenesisConfig` in the shared test helper or document clearly that the helper funds an account only and does not initialize treasury pallet storage.

EQ-QNT-TREASURY-O-01: Changing the treasury account only redirects future reward flow

Updating the treasury account changes where future rewards are sent, but it does not migrate any balance that has already accumulated in the previously configured account. That behaviour may be intentional, but it is operationally significant enough that it should be stated clearly.

The setter simply updates storage and emits an event:

pallets/treasury/src/lib.rs:74-82

```
74 |     /// Set the treasury account. Root only.
75 |     #[pallet::call_index(0)]
76 |     #[pallet::weight(T::WeightInfo::set_treasury_account())]
77 |     pub fn set_treasury_account(origin: OriginFor<T>, account: T::AccountId) -> DispatchResult {
78 |       ensure_root(origin)?;
79 |       TreasuryAccount::<T>::put(&account);
80 |       Self::deposit_event(Event::TreasuryAccountUpdated { new_account: account });
81 |       Ok(())
82 |     }
```

As written, the extrinsic behaves more like a routing update than an account migration. The emitted event also includes only `new_account`, so event-only consumers cannot reconstruct the full transition without separately reading historical storage. Users or operators expecting an administrative "move the treasury" operation may otherwise assume that existing funds are carried across automatically.

Recommendation
Document the current semantics explicitly. If a full account migration is desired, consider adding `old_account: T::AccountId` to the `TreasuryAccountUpdated` event so event-only consumers can reconstruct each transition without reading historical storage.

EQ-QNT-TREASURY-O-02: Treasury reward portion is stored as a `u8` despite being used as a percentage type

The treasury reward share is stored as a plain `u8`, manually constrained to the range `0..=100`, and then converted to a percentage-like type whenever it is used. The current implementation is functional, but the stored type does not make the intended percentage semantics obviously correct.

The pallet stores and validates the value as a raw byte:

pallets/treasury/src/lib.rs:42-117

```
42 |   pub struct GenesisConfig<T: Config> {
43 |     pub treasury_account: T::AccountId,
44 |     pub treasury_portion: u8,
45 |   }
```

```
56 |   #[pallet::genesis_build]
57 |   impl<T: Config> BuildGenesisConfig for GenesisConfig<T> {
58 |     fn build(&self) {
59 |       assert!(self.treasury_portion <= 100, "Treasury portion must be 0-100");
60 |       TreasuryAccount::<T>::put(self.treasury_account.clone());
61 |       TreasuryPortion::<T>::put(self.treasury_portion);
62 |     }
```

```
 63 |   }
```

```
115 |     pub fn portion() -> u8 {
116 |       TreasuryPortion::<T>::get()
117 |     }
```

But the downstream reward calculation immediately treats it as a percentage type:

pallets/mining-rewards/src/lib.rs:150-152

```
150 |       let treasury_portion = T::Treasury::portion();
151 |       let treasury_reward =
152 |         Permill::from_percent(u32::from(treasury_portion)).mul_floor(total_reward);
```

Using a more direct percentage representation would improve readability and make it obvious that the value is intended as a rate rather than an arbitrary byte. It would also make fractional reward shares easier to support in the future if the pallet ever needs more granular configuration.

Recommendation
Replace the raw `u8` with a dedicated percentage type. Either store the reward portion as a `Permill`, which has higher precision but requires a storage migration, or store it as a `Percent` and accept the coarser 1% precision.

EQ-QNT-TREASURY-O-03: The standalone treasury pallet adds boilerplate without clear separation of responsibilities

The treasury pallet currently exposes only storage, getters, and administrative setters consumed by the mining-rewards pallet. A separate abstraction is not inherently inappropriate, but the separation does not yet buy much architectural clarity on its own.

If the intent is to grow the pallet into a broader treasury subsystem with budgets, spending, or governance, then that separation is justified. If not, the present design may be more indirection than the codebase currently needs.

Recommendation
Either document the longer-term role of the treasury pallet or consider inlining the current configuration surface into the part of the runtime that actually consumes it.

EQ-QNT-TREASURY-O-04: Treasury-side mint failures are silent on-chain

The mining-rewards pallet treats treasury mint failure as a log-only error. That does not establish permanent reward destruction in the current runtime, but it does leave operators with no on-chain signal if the treasury reward path ever fails.

Within `mint_reward`, treasury minting happens in two places: the `Some(miner)` arm falls back to treasury if minting to the miner fails, and the `None` arm mints to treasury directly.

pallets/mining-rewards/src/lib.rs:211-219

```
211 |     fn mint_reward(maybe_miner: Option<T::AccountId>, reward: BalanceOf<T>) {
212 |       if reward.is_zero() {
213 |         return;
214 |       }
215 |
216 |       let mint_account = T::MintingAccount::get();
217 |       let treasury = T::Treasury::account_id();
218 |
219 |       match maybe_miner {
```

In the `Some(miner)` arm, the pallet first tries to mint to the miner, then falls back to treasury and emits a redirect event if that fallback succeeds:

pallets/mining-rewards/src/lib.rs:220-260

```
220 |         Some(miner) => {
221 |           match T::Currency::mint_into(&miner, reward) {
---
231 |             Err(e) => {
232 |               log::warn!(
233 |                 target: "mining-rewards",
234 |                 "Failed to mint {:?} to miner {:?}: {:?}, redirecting to treasury",
235 |                 reward, miner, e
236 |               );
237 |               // Fallback: redirect to treasury
238 |               match T::Currency::mint_into(&treasury, reward) {
239 |                 Ok(_) => {
240 |                   T::ProofRecorder::record_transfer_proof(
241 |                     None, // Native token
242 |                     mint_account,
243 |                     treasury,
244 |                     reward,
245 |                   );
246 |                   Self::deposit_event(Event::MinerRewardRedirected {
247 |                     miner,
248 |                     reward,
249 |                   });
---
251 |                 Err(e2) => {
252 |                   log::error!(
```

```
253 |                     target: "mining-rewards",
254 |                     "Failed to redirect {:?} to treasury: {:?}",
255 |                     reward, e2
256 |                   );
257 |                 },
258 |               }
259 |             },
260 |           }
```

In the `None` arm, the reward is meant for treasury directly, and failure only produces a log message with no on-chain event:

pallets/mining-rewards/src/lib.rs:262-281

```
262 |         None => {
263 |           match T::Currency::mint_into(&treasury, reward) {
264 |             Ok(_) => {
    ---------------------------------------------------------------
271 |               Self::deposit_event(Event::TreasuryRewarded { reward });
272 |             },
273 |             Err(e) => {
274 |               log::error!(
275 |                 target: "mining-rewards",
276 |                 "Failed to mint {:?} to treasury: {:?}",
277 |                 reward, e
278 |               );
279 |             },
280 |           }
281 |         },
```

Impact boundary

The treasury account is configured in normal runtime presets, so this does not establish present reward loss in the audited configuration. The issue is narrower and more operational: if the treasury mint path ever does fail because of a future runtime change, bad configuration, or a currency invariant violation, downstream observers will not learn about it from on-chain state.

That makes the failure mode difficult to monitor and difficult to distinguish from “no treasury reward was due this block” without privileged log access. For an issuance path, that is a weak observability contract even if the path is not expected to fail today.

Recommendation

Emit a dedicated treasury-mint-failed event, or enforce the treasury mint path as an explicit runtime invariant during startup or upgrade validation.

4. MULTISIG

EQ-QNT-MULTISIG-F-01: Dispatched `RuntimeCall` execution is not charged in the multisig fee model

`execute` charges for multisig bookkeeping, not for the `RuntimeCall` that it dispatches. A multisig can therefore route expensive runtime work through a fee model that only accounts for loading, checking, decoding, and deleting the stored proposal.

The extrinsic computes actual weight from the encoded call length alone:

pallets/multisig/src/lib.rs:984-1026

```
 984 |       // Check if executor is a signer (1 read: Multisigs)
 985 |       let multisig_data = Multisigs::<T>::get(&multisig_address).ok_or_else(|| {
 986 |         DispatchErrorWithPostInfo {
 987 |           post_info: PostDispatchInfo {
 988 |             actual_weight: Some(T::DbWeight::get().reads(1)),
 989 |             pays_fee: Pays::Yes,
 990 |           },
 991 |           error: Error::<T>::MultisigNotFound.into(),
 992 |         }
 993 |       })?;
 994 |       if !multisig_data.signers.contains(&executor) {
 995 |         return Self::err_with_weight(Error::<T>::NotASigner, 1);
 996 |       }
 997 |
 998 |       // Get proposal (2 reads: Multisigs + Proposals)
 999 |       let proposal =
1000 |         Proposals::<T>::get(&multisig_address, proposal_id).ok_or_else(|| {
1001 |           DispatchErrorWithPostInfo {
1002 |             post_info: PostDispatchInfo {
1003 |               actual_weight: Some(T::DbWeight::get().reads(2)),
1004 |               pays_fee: Pays::Yes,
1005 |             },
1006 |             error: Error::<T>::ProposalNotFound.into(),
1007 |           }
1008 |         })?;
```

```
1021 |       // Calculate actual weight based on real call size
1022 |       let actual_call_size = proposal.call.len() as u32;
1023 |       let actual_weight = <T as Config>::WeightInfo::execute(actual_call_size);
1024 |
1025 |       // Execute the proposal
1026 |       Self::do_execute(multisig_address, proposal_id, proposal)?;
```

But the internal execution path dispatches the decoded runtime call without feeding that dispatch weight back into fee accounting:

pallets/multisig/src/lib.rs:1291-1312

```
1291 |     fn do_execute(
1292 |       multisig_address: T::AccountId,
1293 |       proposal_id: u32,
1294 |       proposal: ProposalDataOf<T>,
1295 |     ) -> DispatchResult {
1296 |       // CHECKS: Decode the call (validation)
1297 |       let call = <T as Config>::RuntimeCall::decode(&mut &proposal.call[..])
1298 |         .map_err(|_| Error::<T>::InvalidCall)?;
---------------------------------------------------------------------------------
1311 |       let result =
1312 |         call.dispatch(frame_system::RawOrigin::Signed(multisig_address.clone()).into());
```

Concrete impact

The gap is easiest to see with a proposal that stores a relatively compact call, but executes substantial runtime work when approved. A `utility::batch` carrying many balance or asset operations, for example, still reaches `execute` through a weight model that scales with encoded call length. The wrapper path therefore charges roughly for "read multisig, read proposal, remove proposal" plus a byte-length slope, while the real dispatch can consume far more weight once the decoded `RuntimeCall` starts mutating storage.

This undercharges expensive inner calls, weakens fee attribution, and leaves block-cost accounting out of line with the work actually being performed.

Comparison to upstream

Substrate's standard `as_multi` pattern treats this as a first-class interface concern. The caller supplies a maximum call weight, the pallet compares that bound against the decoded call's declared dispatch weight, and the extrinsic returns post-dispatch weight information so fees can be refunded from a bounded worst case. The current pallet skips all three steps: `execute` has no max-weight parameter, it does not check the decoded call's `get_dispatch_info().weight`, and it returns only the wrapper's size-derived weight model after dispatch.

Recommendation

Reshape `execute` around the same control points. Accept an explicit maximum call-weight parameter, validate it against the decoded call's dispatch info before execution, include that bound in the extrinsic's worst-case weight declaration, and return actual post-dispatch weight so excess fees can be refunded. If the pallet wants a simpler interface than upstream `as_multi`, it still needs some equivalent bound-and-refund mechanism. Charging only for multisig bookkeeping is not enough.

EQ-QNT-MULTISIG-F-02: Dynamic proposal fee formula floors too early

The pallet's proposal fee grows with the number of signers, but the current calculation applies `mul_floor` before multiplying by signer count. This is more conservative than the surrounding comments suggest and can flatten the fee increase entirely for small base fees.

The current fee calculation does the flooring step per signer:

pallets/multisig/src/lib.rs:600-609

```
600 |       // Calculate dynamic fee based on number of signers
601 |       // Fee = Base + (Base * SignerCount * StepFactor)
602 |       let base_fee = T::ProposalFee::get();
603 |       let step_factor = T::SignerStepFactor::get();
604 |
605 |       // Calculate extra fee: (Base * Factor) * Count
606 |       // mul_floor returns the part of the fee corresponding to the percentage
607 |       let fee_increase_per_signer = step_factor.mul_floor(base_fee);
608 |       let total_increase = fee_increase_per_signer.saturating_mul(signers_count.into());
609 |       let fee = base_fee.saturating_add(total_increase);
```

For example, a base fee of `99` with a `1%` step factor and `100` signers yields a per-signer increment of zero after early flooring, producing no increase at all. That does not match the intuitive "base fee plus percentage growth per signer" interpretation.

```
base_fee = 99
step_factor = 1%
signers_count = 100
per_signer = floor(99 * 1%) = floor(0.99) = 0
total_increase = 0 * 100 = 0
```

Recommendation

Reorder the arithmetic so the multiplication by signer count happens before the final flooring step, subject to the usual saturation checks. For example:

```
let base_fee = T::ProposalFee::get();
let step_factor = T::SignerStepFactor::get();

// Calculate dynamic fee based on number of signers
// Multiplier = Base * SignerCount
let multiplier = base_fee.saturating_mul(signers_count.into());
// TotalIncrease = floor(StepFactor * Multiplier) = floor(StepFactor * Base *
SignerCount)
let total_increase = step_factor.mul_floor(multiplier);
// Fee = Base + TotalIncrease
//     = Base + floor(StepFactor * Base * SignerCount)
let fee = base_fee.saturating_add(total_increase);
```

Under the same inputs this yields:

```
base_fee = 99
step_factor = 1%
signers_count = 100
multiplier = 99 * 100 = 9900
total_increase = floor(1% * 9900) = 99
```

EQ-QNT-MULTISIG-F-03: Several secondary multisig paths do not return accurate weight information

The multisig pallet uses more than one pattern for retrieving and validating multisig state. Some paths return weighted errors precisely, while others collapse back to ordinary `DispatchError` handling or charge a worst-case path even when the actual work is much cheaper.

Examples

`approve` uses an explicit weighted failure for missing multisigs and non-signers:

pallets/multisig/src/lib.rs:712-723

```
712 |       let multisig_data = Multisigs::<T>::get(&multisig_address).ok_or_else(|| {
713 |         DispatchErrorWithPostInfo {
714 |           post_info: PostDispatchInfo {
715 |             actual_weight: Some(T::DbWeight::get().reads(1)),
716 |             pays_fee: Pays::Yes,
717 |           },
718 |           error: Error::<T>::MultisigNotFound.into(),
719 |         }
720 |       })?;
721 |       if !multisig_data.signers.contains(&approver) {
722 |         return Self::err_with_weight(Error::<T>::NotASigner, 1);
723 |       }
```

But `ensure_is_signer`, which is used by `remove_expired` and `claim_deposits`, drops back to a plain `DispatchError` path:

pallets/multisig/src/lib.rs:1180-1188

```
1180 |     fn ensure_is_signer(
1181 |       multisig_address: &T::AccountId,
1182 |       account: &T::AccountId,
1183 |     ) -> Result<MultisigDataOf<T>, DispatchError> {
1184 |       let multisig_data =
1185 |         Multisigs::<T>::get(multisig_address).ok_or(Error::<T>::MultisigNotFound)?;
1186 |       ensure!(multisig_data.signers.contains(account), Error::<T>::NotASigner);
1187 |       Ok(multisig_data)
1188 |     }
```

And `remove_expired` is still declared with worst-case call size and no post-dispatch refund path:

pallets/multisig/src/lib.rs:859-864

```
859 |     #[pallet::weight(<T as Config>::WeightInfo::remove_expired(T::MaxCallSize::get()))]
860 |     pub fn remove_expired(
861 |       origin: OriginFor<T>,
862 |       multisig_address: T::AccountId,
863 |       proposal_id: u32,
864 |     ) -> DispatchResult {
```

The same underlying gap appears in several places:

- `propose`, `approve`, and `execute` perform raw reads with explicit weighted failures.
- `remove_expired` and `claim_deposits` use a helper that does not preserve the same weight information for expected signer and existence failures.
- `remove_expired` also charges against `T::MaxCallSize::get()` even on its ordinary successful path.
- `approve_dissolve` similarly charges its most expensive path even when it fails early.
- Deferred decode failures in `execute` also fall back to less accurate charging.

None of these issues amount to a direct exploit in isolation, but together they make fee behaviour harder to reason about and can undercharge or overcharge ordinary application failures.

Recommendation
Standardise the pallet on one helper pattern that preserves accurate expected-failure charging and, where relevant, returns post-dispatch weight information for the ordinary successful path as well.

EQ-QNT-MULTISIG-F-04: `claim_deposits` weight and benchmark assumptions do not match the stated worst case

The extrinsic weight annotation for `claim_deposits` hard-codes `MaxTotalProposalsInStorage / 2` as the supposed worst-case cleaned count, but there is no clear reason that should be the upper bound. The benchmark itself permits `iterated == cleaned`, so the obvious worst case is that both reach `MaxTotalProposalsInStorage`.

The current annotation is:

pallets/multisig/src/lib.rs:917-921

```
917 |     #[pallet::weight(<T as Config>::WeightInfo::claim_deposits(
918 |     T::MaxTotalProposalsInStorage::get(),  // Worst-case iterated
919 |     T::MaxTotalProposalsInStorage::get().saturating_div(2),  // Worst-case cleaned
920 |     T::MaxCallSize::get()  // Worst-case avg call size
921 |   ))]
```

And the benchmark computes the number of cleaned proposals as `min(i, r)`:

pallets/multisig/src/benchmarking.rs:385-392

```
385 |   #[benchmark]
386 |   fn claim_deposits(
387 |     i: Linear<1, { T::MaxTotalProposalsInStorage::get() }>,
388 |     r: Linear<1, { T::MaxTotalProposalsInStorage::get() }>,
389 |     c: Linear<0, { T::MaxCallSize::get().saturating_sub(100) }>,
390 |   ) -> Result<(), BenchmarkError> {
391 |     let cleaned_target = (r as u32).min(i);
392 |     let total_proposals = i;
```

That means the measured cleanup work does not really exercise `r` as an independent dimension across the full range. Once `r > i`, increasing `r` stops changing the measured work and the curve flattens to `i`.

This does not suggest an immediate exploit, but it does mean the weight annotation and the benchmark are claiming more precision than they actually have.

Recommendation
Use the real worst-case cleaned count in the weight annotation, and align the benchmark shape with the cost model it is supposed to justify. In practice, FRAME's benchmarking interface does not make it easy to express the full valid domain here with dependent inputs such as `r <= i`. A pragmatic improvement would be to benchmark a constrained region where that relationship already holds, remove the clamp distortion from the sampled points, and document the remaining gap explicitly rather than implying the current model measures those dimensions independently.

EQ-QNT-MULTISIG-F-05: `DepositsClaimed.total_returned` can drift from the amounts actually unreserved

The `claim_deposits` summary event reports `total_returned` using the current configured proposal deposit, but the cleanup path actually unreserves the stored per-proposal deposit values. After a runtime upgrade changes `ProposalDeposit`, those two quantities can diverge.

The summary event is computed from the current configuration value:

pallets/multisig/src/lib.rs:923-945

```
923 |     pub fn claim_deposits(
924 |       origin: OriginFor<T>,
925 |       multisig_address: T::AccountId,
926 |     ) -> DispatchResultWithPostInfo {
927 |       let caller = ensure_signed(origin)?;
928 |
929 |       Self::ensure_is_signer(&multisig_address, &caller)?;
930 |
931 |       let (cleaned, total_proposals_iterated, total_call_bytes) =
932 |         Self::cleanup_proposer_expired(&multisig_address, &caller, &caller);
933 |
934 |       let deposit_per_proposal = T::ProposalDeposit::get();
935 |       let total_returned = deposit_per_proposal.saturating_mul(cleaned.into());
936 |
937 |       // Emit summary event
938 |       Self::deposit_event(Event::DepositsClaimed {
939 |         multisig_address: multisig_address.clone(),
940 |         claimer: caller,
941 |         total_returned,
942 |         proposals_removed: cleaned,
943 |         multisig_removed: false,
944 |       });
945 |
```

But the cleanup path removes each proposal using the deposit that was stored with that proposal:

pallets/multisig/src/lib.rs:1216-1273

```
1216 |             total_call_bytes += proposal.call.len() as u32;
1217 |
1218 |             // Only proposer's expired proposals (Active or Approved)
1219 |             if proposal.proposer == *proposer &&
1220 |               (proposal.status == ProposalStatus::Active ||
1221 |                 proposal.status == ProposalStatus::Approved) &&
1222 |               current_block > proposal.expiry
1223 |             {
1224 |               Some((proposal_id, proposal.proposer, proposal.deposit))
1225 |             } else {
1226 |               None
1227 |             }
1228 |           })
1229 |           .collect();
1230 |
1231 |       let cleaned = expired_proposals.len() as u32;
1232 |
1233 |       // Remove proposals and emit events
1234 |       for (proposal_id, expired_proposer, deposit) in expired_proposals {
1235 |         Self::remove_proposal_and_return_deposit(
1236 |           multisig_address,
1237 |           proposal_id,
1238 |           &expired_proposer,
1239 |           deposit,
```

```
1255 |     fn remove_proposal_and_return_deposit(
1256 |       multisig_address: &T::AccountId,
1257 |       proposal_id: u32,
1258 |       proposer: &T::AccountId,
1259 |       deposit: BalanceOf<T>,
1260 |     ) {
1261 |       // Remove from storage
1262 |       Proposals::<T>::remove(multisig_address, proposal_id);
1263 |
1264 |       // Decrement proposal counters
1265 |       Multisigs::<T>::mutate(multisig_address, |maybe_data| {
1266 |         if let Some(ref mut data) = maybe_data {
1267 |           data.active_proposals = data.active_proposals.saturating_sub(1);
1268 |           if let Some(count) = data.proposals_per_signer.get_mut(proposer) {
1269 |             *count = count.saturating_sub(1);
1270 |             // Remove entry if count reaches 0 to save storage
1271 |             if *count == 0 {
1272 |               data.proposals_per_signer.remove(proposer);
1273 |             }
```

This is not a fund-loss issue by itself. The underlying balance updates still use the stored deposit amounts. The problem is that the emitted event can stop being an accurate record of what was actually returned once runtime configuration changes over time.

Recommendation
Either compute `total_returned` from the actual stored deposits removed in this call, or document the field as a nominal value rather than an exact returned amount.

EQ-QNT-MULTISIG-F-06: Dissolution can be prevented by sending trivial native balances to the multisig

The dissolution path requires the multisig account's native balance to be exactly zero before the final approval can succeed. Because the account address is public and deterministic, any third party can send a trivial amount of native currency to the multisig and block dissolution until the signers notice and clear the dust balance themselves.

`approve_dissolve` rejects dissolution whenever the native balance is non-zero:

pallets/multisig/src/lib.rs:1065-1067

```
1065 |       // 4. Check if account balance is zero
1066 |       let balance = T::Currency::total_balance(&multisig_address);
1067 |       ensure!(balance.is_zero(), Error::<T>::MultisigAccountNotZero);
```

This creates a low-skill griefing path. An external account does not need any multisig privileges to keep the account from reaching the “clean” state required for dissolution; it only needs to send a small amount of currency to the publicly known multisig address.

Recommendation
Avoid requiring an exact zero native balance for dissolution, or provide an explicit sweep path that lets signers clear unexpected dust balances as part of the dissolution flow.

EQ-QNT-MULTISIG-F-07: Dissolution ignores non-native assets and can strand them

The multisig can schedule asset transfers, but the dissolution path only checks the native currency balance before removing multisig storage. As a result, non-native assets held by the multisig are not accounted for during dissolution and can become stranded once the multisig record is removed.

The high-security runtime whitelist permits asset-transfer scheduling through the multisig:

runtime/src/configs/mod.rs:574-580

```
574 |   fn is_whitelisted(call: &RuntimeCall) -> bool {
575 |     matches!(
576 |       call,
577 |       RuntimeCall::ReversibleTransfers(
578 |         pallet_reversible_transfers::Call::schedule_transfer { .. }
579 |       ) | RuntimeCall::ReversibleTransfers(
580 |         pallet_reversible_transfers::Call::schedule_asset_transfer { .. }
```

But dissolution only checks the native currency balance and then removes multisig storage:

pallets/multisig/src/lib.rs:1065-1099

```
1065 |       // 4. Check if account balance is zero
1066 |       let balance = T::Currency::total_balance(&multisig_address);
1067 |       ensure!(balance.is_zero(), Error::<T>::MultisigAccountNotZero);
---
1094 |         // Remove multisig from storage
1095 |         Multisigs::<T>::remove(&multisig_address);
1096 |         DissolveApprovals::<T>::remove(&multisig_address);
1097 |
1098 |         // Return deposit to creator
1099 |         T::Currency::unreserve(&creator, deposit);
```

This creates a real asset-accessibility risk rather than a documentation issue. Native funds block dissolution, but non-native assets do not, so the multisig can be removed while other asset balances remain associated with that account.

Recommendation
At minimum, prevent dissolution while supported non-native asset balances remain. If dissolution with residual assets is intended, the pallet should define an explicit asset-recovery path.

EQ-QNT-MULTISIG-O-01: High-security validation and call decoding are deferred later than necessary

The multisig pallet defers some checks until later in the flow, but the resulting ordering is hard to justify.

The proposal path performs the high-security check and decode before later proposal-limit and expiry checks:

pallets/multisig/src/lib.rs:552-598

```
552 |       let is_high_security = T::HighSecurity::is_high_security(&multisig_address);
553 |       if is_high_security {
554 |         let decoded_call =
555 |           <T as Config>::RuntimeCall::decode(&mut &call[..]).map_err(|_| {
556 |             DispatchErrorWithPostInfo {
557 |               post_info: PostDispatchInfo {
558 |                 actual_weight: Some(T::DbWeight::get().reads(2)),
559 |                 pays_fee: Pays::Yes,
560 |               },
561 |               error: Error::<T>::InvalidCall.into(),
562 |             }
563 |           })?;
    ---
577 |       ensure!(
578 |         multisig_data.active_proposals < T::MaxTotalProposalsInStorage::get(),
579 |         Error::<T>::TooManyProposalsInStorage
580 |       );
    ---
584 |       let max_proposals_per_signer =
585 |         T::MaxTotalProposalsInStorage::get().saturating_div(signers_count);
586 |       let proposer_current_count =
587 |         multisig_data.proposals_per_signer.get(&proposer).copied().unwrap_or(0);
588 |       ensure!(
589 |         proposer_current_count < max_proposals_per_signer,
590 |         Error::<T>::TooManyProposalsPerSigner
591 |       );
    ---
594 |       ensure!(expiry > current_block, Error::<T>::ExpiryInPast);
    ---
598 |       ensure!(expiry <= max_expiry, Error::<T>::ExpiryTooFar);
```

Ordinary calls, by contrast, are decoded only much later during execution:

pallets/multisig/src/lib.rs:1291-1298

```
1291 |     fn do_execute(
1292 |       multisig_address: T::AccountId,
1293 |       proposal_id: u32,
1294 |       proposal: ProposalDataOf<T>,
1295 |     ) -> DispatchResult {
1296 |       // CHECKS: Decode the call (validation)
1297 |       let call = <T as Config>::RuntimeCall::decode(&mut &proposal.call[..])
1298 |         .map_err(|_| Error::<T>::InvalidCall)?;
```

Two patterns are especially relevant:

- the high-security check is performed before some cheaper proposal-count and expiry checks even though it could be delayed without affecting the happy path,
- only high-security multisigs decode the stored call up front, so ordinary decode failures are discovered later during execution and fall back to less accurate weight charging.

This may be an intentional tradeoff, but it leaves error behaviour less predictable and postpones malformed-call detection.

Recommendation
Either validate and decode the call earlier in a consistent place, or document more clearly why the current ordering is preferable.

EQ-QNT-MULTISIG-O-02: Signer normalisation and duplicate checks are split across multiple layers

Signer normalisation is currently distributed across more than one layer of the multisig API. This makes the code harder to read and leaves related invariants only partially centralised.

Signer normalisation is split between multisig creation and address derivation:

pallets/multisig/src/lib.rs:437-449

```
437 |       // Sort signers for duplicate check and storage
438 |       let mut sorted_signers = signers.clone();
439 |       sorted_signers.sort();
440 |
441 |       // Check for duplicate signers
442 |       for i in 1..sorted_signers.len() {
443 |         ensure!(sorted_signers[i] != sorted_signers[i - 1], Error::<T>::DuplicateSigner);
444 |       }
445 |
446 |       // Generate deterministic multisig address
447 |       // Note: derive_multisig_address() will sort internally, but we already have sorted
448 |       // for duplicate check, so we pass sorted to avoid double sorting
449 |       let multisig_address = Self::derive_multisig_address(&sorted_signers, threshold, nonce);
```

pallets/multisig/src/lib.rs:1136-1147

```
1136 |     /// The address is computed as: hash(pallet_id || sorted_signers || threshold || nonce)
1137 |     /// Signers are automatically sorted internally for deterministic results.
1138 |     /// This allows users to pre-compute the address before creating the multisig.
1139 |     pub fn derive_multisig_address(
1140 |       signers: &[T::AccountId],
1141 |       threshold: u32,
1142 |       nonce: u64,
1143 |     ) -> T::AccountId {
1144 |       // Sort signers for deterministic address generation
1145 |       // User doesn't need to worry about order
1146 |       let mut sorted_signers = signers.to_vec();
1147 |       sorted_signers.sort();
```

- signer sorting is performed for duplicate detection in `create_multisig`, then effectively repeated in `derive_multisig_address`,
- duplicate signer checks are not centralised in the same place as signer normalisation.

This is a design-quality issue rather than a direct bug, but the pallet would be easier to maintain if signer ordering and duplicate-check guarantees were made more explicit.

Recommendation
Push signer normalisation and duplicate checking into a single authoritative boundary.

EQ-QNT-MULTISIG-O-03: The extrinsic call boundary is looser than the pallet's own limits require

The `propose` extrinsic still accepts a raw `Vec<u8>` even though the pallet already knows the relevant call-size bound at that API edge.

The extrinsic accepts an unbounded byte vector for the call payload:

pallets/multisig/src/lib.rs:520-525

```
520 |     pub fn propose(
521 |       origin: OriginFor<T>,
522 |       multisig_address: T::AccountId,
523 |       call: Vec<u8>,
524 |       expiry: BlockNumberFor<T>,
525 |     ) -> DispatchResultWithPostInfo {
```

This is a design-quality issue rather than a direct bug, but the API would communicate its contract more clearly if it rejected oversized inputs through the type boundary itself.

Recommendation
Prefer a bounded call input type at the extrinsic edge where the pallet already knows the relevant limit.

EQ-QNT-MULTISIG-O-04: Proposal lifecycle semantics and naming are inconsistent

The multisig state machine is broadly understandable, but several names and comments do not line up well with the behaviour they describe.

Examples
The approval path emits `ProposalReadyToExecute` whenever the proposal status is already `Approved`, not only when the threshold has just been crossed:

pallets/multisig/src/lib.rs:758-781

```
758 |       // Check if threshold is reached - if so, mark as Approved
759 |       if approvals_count >= multisig_data.threshold {
760 |         proposal.status = ProposalStatus::Approved;
761 |       }
762 |
763 |       // Save proposal
764 |       Proposals::<T>::insert(&multisig_address, proposal_id, &proposal);
765 |
766 |       // Emit approval event
767 |       Self::deposit_event(Event::ProposalApproved {
768 |         multisig_address: multisig_address.clone(),
769 |         approver,
770 |         proposal_id,
771 |         approvals_count,
772 |       });
773 |
774 |       // Emit ready-to-execute event if threshold just reached
775 |       if proposal.status == ProposalStatus::Approved {
776 |         Self::deposit_event(Event::ProposalReadyToExecute {
777 |           multisig_address,
778 |           proposal_id,
779 |           approvals_count,
780 |         });
781 |       }
```

The cancellation check and the associated error text are also not perfectly aligned:

pallets/multisig/src/lib.rs:820-824

```
820 |       if proposal.status != ProposalStatus::Active &&
821 |         proposal.status != ProposalStatus::Approved
822 |       {
823 |         return Self::err_with_weight(Error::<T>::ProposalNotActive, 1);
824 |       }
```

pallets/multisig/src/lib.rs:346-393

```
346 |   #[pallet::error]
347 |   pub enum Error<T> {
```

```
392 |     /// Proposal is not active (already executed or cancelled)
393 |     ProposalNotActive,
```

Examples include:

- `Event::ProposalApproved` means "approved by a signer", not necessarily that the proposal's status has become `Approved`,
- `ProposalReadyToExecute` can be emitted repeatedly once the approval threshold has already been reached,
- the `ProposalNotActive` error text does not describe all of the non-cancellable states cleanly,
- the `Executed` and `Cancelled` `ProposalStatus` variants are currently unused.

These mismatches increase the amount of context required to understand proposal state and make event streams less self-explanatory than they could be.

Recommendation
Align event names, comments, and enum variants more closely with the actual state transitions the code performs.

EQ-QNT-MULTISIG-O-05: Proposal identifiers saturate instead of failing explicitly at exhaustion

The pallet's proposal identifier is implemented as a monotonic `u32`, but it is advanced with `saturating_add` rather than with an explicit exhaustion check.

Proposal creation currently increments the identifier this way:

pallets/multisig/src/lib.rs:631-645

```
631 |       let proposal_id = Multisigs::<T>::try_mutate(
632 |         &multisig_address,
633 |         |maybe_multisig| -> Result<u32, DispatchError> {
634 |           let multisig = maybe_multisig.as_mut().ok_or(Error::<T>::MultisigNotFound)?;
635 |           let nonce = multisig.proposal_nonce;
636 |           multisig.proposal_nonce = nonce.saturating_add(1);
637 |           multisig.active_proposals = multisig.active_proposals.saturating_add(1);
638 |           let count = multisig.proposals_per_signer.get(&proposer).copied().unwrap_or(0);
639 |           multisig
640 |             .proposals_per_signer
641 |             .try_insert(proposer.clone(), count.saturating_add(1))
```

```
642 |             .map_err(|_| Error::<T>::TooManySigners)?;
643 |           Ok(nonce)
644 |         },
645 |       )?;
```

This is a theoretical edge case rather than a practical near-term failure mode. Still, if the identifier ever reached `u32::MAX`, it would stop advancing cleanly and begin reusing the terminal value instead of failing explicitly.

Recommendation
Treat identifier exhaustion as an explicit case: either reject further proposals once the counter is exhausted or move to an identifier type and lifecycle model that makes the intended bound clearer.

EQ-QNT-MULTISIG-O-06: Proposal bookkeeping and mutation paths are more complicated than they need to be

Several multisig state updates are technically serviceable but rely on more cached bookkeeping and broader mutation interfaces than the surrounding logic seems to need.

Examples
Proposal creation updates a cached active-proposal counter and per-signer counts inline:

pallets/multisig/src/lib.rs:631-645

```
631 |       let proposal_id = Multisigs::<T>::try_mutate(
632 |         &multisig_address,
633 |         |maybe_multisig| -> Result<u32, DispatchError> {
634 |           let multisig = maybe_multisig.as_mut().ok_or(Error::<T>::MultisigNotFound)?;
635 |           let nonce = multisig.proposal_nonce;
636 |           multisig.proposal_nonce = nonce.saturating_add(1);
637 |           multisig.active_proposals = multisig.active_proposals.saturating_add(1);
638 |           let count = multisig.proposals_per_signer.get(&proposer).copied().unwrap_or(0);
639 |           multisig
640 |             .proposals_per_signer
641 |             .try_insert(proposer.clone(), count.saturating_add(1))
642 |             .map_err(|_| Error::<T>::TooManySigners)?;
643 |           Ok(nonce)
644 |         },
645 |       )?;
```

Proposal removal then decrements the same cached bookkeeping through a generic optional mutation:

pallets/multisig/src/lib.rs:1255-1273

```
1255 |     fn remove_proposal_and_return_deposit(
1256 |       multisig_address: &T::AccountId,
1257 |       proposal_id: u32,
1258 |       proposer: &T::AccountId,
1259 |       deposit: BalanceOf<T>,
1260 |     ) {
1261 |       // Remove from storage
1262 |       Proposals::<T>::remove(multisig_address, proposal_id);
1263 |
1264 |       // Decrement proposal counters
1265 |       Multisigs::<T>::mutate(multisig_address, |maybe_data| {
1266 |         if let Some(ref mut data) = maybe_data {
1267 |           data.active_proposals = data.active_proposals.saturating_sub(1);
1268 |           if let Some(count) = data.proposals_per_signer.get_mut(proposer) {
1269 |             *count = count.saturating_sub(1);
1270 |             // Remove entry if count reaches 0 to save storage
1271 |             if *count == 0 {
1272 |               data.proposals_per_signer.remove(proposer);
1273 |             }
```

And one cleanup path still threads both `proposer` and `caller` even though the current use site passes the same account for both:

pallets/multisig/src/lib.rs:927-932

```
927 |       let caller = ensure_signed(origin)?;
928 |
929 |       Self::ensure_is_signer(&multisig_address, &caller)?;
930 |
931 |       let (cleaned, total_proposals_iterated, total_call_bytes) =
932 |         Self::cleanup_proposer_expired(&multisig_address, &caller, &caller);
```

The recurring theme is avoidable bookkeeping complexity:

- the `active_proposals` field in `Multisig` could be derived from `proposals_per_signer.values().sum()`,
- proposal creation and removal use broader mutation surfaces than necessary for invariants that are otherwise expected to hold,
- one cleanup helper accepts both `proposer` and `caller` even though they are always the same in current usage,
- `propose` has grown substantially larger than the neighbouring extrinsics.

None of these items are severe on their own, but they all point in the same direction: the pallet would benefit from a pass focused specifically on reducing state duplication and sharpening naming.

Recommendation
Treat the multisig state model as its own refactor target rather than addressing these items only incidentally while working on unrelated features.

EQ-QNT-MULTISIG-O-07: Dissolution and asset-handling semantics need clearer documentation

Several multisig behaviours look intentional but are insufficiently documented, especially around dissolution, creator responsibilities, and the high-security call model.

Examples
The dissolution path iterates proposals directly:

pallets/multisig/src/lib.rs:1061-1063

```
1061 |       if Proposals::<T>::iter_prefix(&multisig_address).next().is_some() {
1062 |         return Err(Error::<T>::ProposalsExist.into());
1063 |       }
```

The direct replacement is a `multisig_data.active_proposals > 0` check on the already loaded record, rather than a fresh `Proposals::iter_prefix(...).next()` probe.

The runtime whitelist also permits `recover_funds`, which the README's high-security list currently omits:

pallets/multisig/README.md:611-616

1. **Setup:** Multisig account calls `ReversibleTransfers::set_high_security(delay, guardian)`
2. **Propose:** Only whitelisted calls allowed:
    - ✅ `ReversibleTransfers::schedule_transfer`
    - ✅ `ReversibleTransfers::schedule_asset_transfer`
    - ✅ `ReversibleTransfers::cancel`
    - ❌ All other calls → `CallNotAllowedForHighSecurityMultisig` error

runtime/src/configs/mod.rs:574-586

```
574 |   fn is_whitelisted(call: &RuntimeCall) -> bool {
575 |     matches!(
576 |       call,
577 |       RuntimeCall::ReversibleTransfers(
578 |         pallet_reversible_transfers::Call::schedule_transfer { .. }
579 |       ) | RuntimeCall::ReversibleTransfers(
```

```
580 |         pallet_reversible_transfers::Call::schedule_asset_transfer { .. }
581 |       ) | RuntimeCall::ReversibleTransfers(pallet_reversible_transfers::Call::cancel { .. }) |
582 |         RuntimeCall::ReversibleTransfers(
583 |           pallet_reversible_transfers::Call::recover_funds { .. }
584 |         )
585 |     )
586 |   }
```

Examples include:

- the creator who pays the deposit does not need to be a signer,
- `approve_dissolve` checks for remaining proposals with an additional storage iteration even though a cached counter is already available,
- after successful call decode, `execute` deletes the proposal even when the dispatched runtime call fails and reports that inner result through the event stream instead of by returning `Err`,
- the README is stale in several places, including whitelisted-call descriptions and a referenced `MULTISIG_REQ.md` file that does not exist.

These are documentation and operational-clarity issues, but they are exactly the sort of details that matter for administrators and downstream integrators.

Recommendation
Document the intentional behaviours explicitly and tighten the implementation where the current behaviour is only accidental or unnecessarily inefficient.

EQ-QNT-MULTISIG-O-08: The current fee attribution model favors simplicity over deposit-backed cleanup

Under the current design, the caller of each multisig extrinsic pays the transaction fee. For most paths this is easy to understand. Some operations, however, have a different economic character.

In particular, `remove_expired`, `execute`, and parts of `approve_dissolve` could instead be paid out of already reserved deposits rather than by whichever signer or caller happens to trigger the cleanup step. That would make the fee model more complicated, but it would also align costs more closely with the proposal that created them.

Recommendation
Keep the current model if simplicity is the priority, but record the tradeoff explicitly so this does not continue to look like an accidental omission.

EQ-QNT-MULTISIG-O-09: Several proposal lifecycle edge cases lack direct test coverage

The multisig pallet still has a substantial set of missing tests. The most important gaps concern proposal lifecycle edge cases and failure-path accounting rather than ordinary success cases.

Representative examples include:

- approving a proposal that is already fully approved,
- saturation of the proposal identifier counter,
- re-entrant execution through a proposal that dispatches back into the multisig pallet,
- immediate dissolution with `threshold = 1`,
- failed deposit reservation during `create_multisig` or `propose`.

The full candidate list is included in Appendix A.

Recommendation
Use the Appendix A checklist as a first-pass test backlog before making further behavioural changes to the pallet.

EQ-QNT-MULTISIG-O-10: Legacy balance traits are used where newer fungible interfaces would be a better fit

The multisig pallet still expresses fee withdrawal and deposit reservation through the older `Currency` and `ReservableCurrency` traits. The current code works, but it keeps the pallet on an older balance-handling abstraction even though its usage is relatively direct.

The pallet imports the legacy balance traits:

pallets/multisig/src/lib.rs:125-132

```
125 |   use frame_support::{
126 |     dispatch::{
127 |       DispatchErrorWithPostInfo, DispatchResult, DispatchResultWithPostInfo, GetDispatchInfo,
128 |       Pays, PostDispatchInfo,
129 |     },
130 |     pallet_prelude::*,
131 |     traits::{Currency, ReservableCurrency},
132 |     PalletId,
```

And uses them directly for fee withdrawal and deposit reservation:

pallets/multisig/src/lib.rs:457-469

```
457 |       // Charge non-refundable fee (burned)
458 |       let fee = T::MultisigFee::get();
459 |       let _ = T::Currency::withdraw(
460 |         &creator,
461 |         fee,
462 |         frame_support::traits::WithdrawReasons::FEE,
463 |         frame_support::traits::ExistenceRequirement::KeepAlive,
464 |       )
465 |       .map_err(|_| Error::<T>::InsufficientBalance)?;
466 |
467 |       // Reserve deposit from creator (will be returned on dissolve)
468 |       let deposit = T::MultisigDeposit::get();
469 |       T::Currency::reserve(&creator, deposit).map_err(|_| Error::<T>::InsufficientBalance)?;
```

This is not a correctness issue on its own, but newer fungible interfaces would express the same operations more directly and align the pallet with current FRAME conventions.

Recommendation
If the pallet is going to evolve further, migrate this balance handling to the newer fungible interfaces so the fee and deposit model is expressed through the current abstraction layer.

EQ-QNT-MULTISIG-O-11: Threshold-one multisigs introduce a special-case proposal path

A multisig with one signer and threshold one is currently valid. That may be an acceptable supported mode, but it means the pallet carries an immediate-approval path that is not needed for ordinary multisigs.

Creation permits any threshold in the range `1..=signers.len()`:

pallets/multisig/src/lib.rs:431-435

```
431 |       // Validate inputs
432 |       ensure!(threshold > 0, Error::<T>::ThresholdZero);
433 |       ensure!(!signers.is_empty(), Error::<T>::NotEnoughSigners);
434 |       ensure!(threshold <= signers.len() as u32, Error::<T>::ThresholdTooHigh);
435 |       ensure!(signers.len() <= T::MaxSigners::get() as usize, Error::<T>::TooManySigners);
```

That case then causes proposals to start in the approved state and emit `ProposalReadyToExecute` immediately:

pallets/multisig/src/lib.rs:629-678

```
629 |       let threshold_met = 1 >= multisig_data.threshold;
```

```
659 |           status: if threshold_met {
```

```
660 |             ProposalStatus::Approved
661 |           } else {
662 |             ProposalStatus::Active
663 |           },
```

```
673 |       if threshold_met {
674 |         Self::deposit_event(Event::ProposalReadyToExecute {
675 |           multisig_address: multisig_address.clone(),
676 |           proposal_id,
677 |           approvals_count: 1,
678 |         });
```

If one-of-one multisigs are a deliberate product mode, then this behaviour is reasonable. If they are not, the pallet is carrying extra state-machine complexity for an edge case that should have been rejected at creation time.

Recommendation
Either document one-of-one multisigs as an explicitly supported mode, or reject them in `create_multisig` and simplify the proposal lifecycle accordingly.

EQ-QNT-MULTISIG-O-12: Per-signer proposal limits rely on an implicit non-zero signer invariant

The per-signer filibuster limit is currently computed with unsigned `saturating_div`. In normal operation `signers_count` should never be zero, but that assumption lives only in earlier validation and stored-state invariants rather than in the calculation itself.

The current calculation is:

pallets/multisig/src/lib.rs:582-585

```
582 |       // Check per-signer proposal limit (filibuster protection)
583 |       // Each signer can have max (TotalLimit / SignersCount) proposals
584 |       let max_proposals_per_signer =
585 |         T::MaxTotalProposalsInStorage::get().saturating_div(signers_count);
```

The current code therefore treats divide-by-zero as impossible without making that assumption explicit at the point where the division happens.

Recommendation
Use `checked_div(signers_count).defensive_unwrap_or(1)` or an equivalent defensive pattern so an impossible zero count is surfaced clearly during development instead of being encoded as an unsigned saturating operation.

5. SCHEDULER

EQ-QNT-SCHEDULER-F-05: Inherited upstream scheduler behaviours remain problematic in this fork

Several of the scheduler behaviours discussed below also exist upstream. That leaves the fork with a real design choice: preserve parity with upstream even where the behaviour is awkward, or diverge intentionally and accept a larger maintenance delta.

Examples

Named-task lookup entries are removed eagerly before task outcome is known:

pallets/scheduler/src/lib.rs:1033-1046

```
1033 |   fn service_task(
```

```
1041 |     // Eagerly remove the name->address lookup. If the task succeeds or gets rescheduled
1042 |     // via a retry, the new address will be re-inserted by place_task. If it fails,
1043 |     // the name is freed.
1044 |     if let Some(ref id) = task.maybe_id {
1045 |       Lookup::<T>::remove(id);
1046 |     }
```

And `do_cancel_named` still succeeds even if the lookup entry points at a missing agenda slot:

pallets/scheduler/src/lib.rs:766-782

```
766 |   fn do_cancel_named(origin: Option<T::PalletsOrigin>, id: TaskName) -> DispatchResult {
767 |     Lookup::<T>::try_mutate_exists(id, |lookup| -> DispatchResult {
768 |       if let Some((when, index)) = lookup.take() {
769 |         let i = index as usize;
770 |         Agenda::<T>::try_mutate(when, |agenda| -> DispatchResult {
771 |           if let Some(s) = agenda.get_mut(i) {
772 |             if let (Some(ref o), Some(ref s)) = (origin, s.borrow()) {
773 |               Self::ensure_privilege(o, &s.origin)?;
774 |             }
775 |             if let Some(ref s) = s {
776 |               Retries::<T>::remove((when, index));
777 |               T::Preimages::drop(&s.call);
778 |             }
779 |             *s = None;
780 |           }
781 |           Ok(())
782 |         })?;
```

The inherited issues include:

- tasks can remain stranded in the agenda forever if preimages are unavailable,
- named-task lookup entries are removed before task outcome is known, so non-first overweight or unavailable named tasks can remain in `Agenda` after losing name-based control,
- later `cancel_named` calls can then either fail with `NotFound` or silently succeed while cleaning up a stale lookup that already points at an empty slot,
- some unused phantom types remain in `Scheduled`.

Practical significance
In the Quantus fork these are control-surface bugs, not just awkward internals. A named task should stay cancellable and inspectable by name until it finishes or is removed. Once the lookup entry is removed early, name-based control and on-chain task state can diverge.

That matters most for named `reversible-transfers` tasks. After a non-permanently overweight or preimage-unavailable failure, the delayed transfer can remain in `Agenda` after losing name-based control. A later `cancel_named` call can return `NotFound`, or succeed only against a stale lookup, while the delayed transfer is still pending. That is hard to reason about operationally and hard to explain to users.

Recommendation
Decide explicitly which upstream quirks are worth preserving and which ones should be treated as fork-specific bugs.

EQ-QNT-SCHEDULER-F-07: Timestamp agenda housekeeping is missing from the base `on_initialize` weight

`on_initialize` charges `service_agendas_base()` once and then services both the block-based and timestamp-based agenda paths. In this fork, the timestamp path is real per-block work, but the base weight was benchmarked only against the block path.

The runtime hook charges one base weight and then executes both agenda domains:

pallets/scheduler/src/lib.rs:352-376

```
352 |     fn on_initialize(now: BlockNumberFor<T>) -> Weight {
    ...
357 |       // Consume base weight for agenda processing
358 |       if weight_counter.try_consume(T::WeightInfo::service_agendas_base()).is_err() {
359 |         return weight_counter.consumed();
360 |       }
361 |
362 |       // Process block-based agendas
363 |       Self::service_block_agendas(&mut weight_counter, now, u32::max_value());
364 |
```

```
365 |       // Process timestamp-based agendas using current system time
366 |       // This ensures no buckets are skipped if block times are longer than bucket intervals
367 |       let current_timestamp = T::TimeProvider::now();
---
370 |       if current_timestamp > T::Moment::zero() {
371 |         Self::service_timestamp_agendas(
372 |           &mut weight_counter,
373 |           current_timestamp,
374 |           u32::max_value(),
375 |         );
376 |       }
```

The timestamp path performs its own bookkeeping reads and writes every time it runs:

pallets/scheduler/src/lib.rs:876-931

```
876 |   fn service_timestamp_agendas(weight: &mut WeightMeter, current_time: T::Moment, max: u32) {
877 |     let normalized_time =
878 |       BlockNumberOrTimestamp::<BlockNumberFor<T>, T::Moment>::Timestamp(current_time)
879 |         .normalize(T::TimestampBucketSize::get())
880 |         .as_timestamp()
881 |         .unwrap_or(T::Moment::zero());
---
887 |     // Start from incomplete timestamp if exists, otherwise from last processed timestamp
888 |     let start_time = if let Some(incomplete) = IncompleteTimestampSince::<T>::take() {
889 |       incomplete
890 |     } else {
891 |       // on the very first processing cycle just set this to current time
892 |       LastProcessedTimestamp::<T>::get().unwrap_or(normalized_time)
893 |     };
---
922 |     // Store incomplete since if needed
923 |     incomplete_since = incomplete_since.min(when);
924 |     if incomplete_since <= normalized_time {
925 |       IncompleteTimestampSince::<T>::put(incomplete_since);
926 |     }
927 |
928 |     // Always update the last processed timestamp to the current block's normalized time.
929 |     // The scheduler's correctness is maintained by `IncompleteTimestampSince`,
930 |     // which is always checked first and takes priority.
931 |     LastProcessedTimestamp::<T>::put(normalized_time);
```

But the benchmark and generated weight only model the block-side `IncompleteBlockSince` path:

pallets/scheduler/src/benchmarking.rs:135-144

```
135 | benchmarks! {
136 |   // `service_block_agendas` when no work is done.
137 |   service_agendas_base {
138 |     let now = BlockNumberFor::<T>::from(BLOCK_NUMBER);
```

```
139 |     IncompleteBlockSince::<T>::put(now - One::one());
140 |   }: {
141 |     Scheduler::<T>::service_block_agendas(&mut WeightMeter::new(), now, 0);
142 |   } verify {
143 |     assert_eq!(IncompleteBlockSince::<T>::get(), Some(now - One::one()));
144 |   }
```

Practical significance
Because the runtime is wired to `Timestamp` with a non-zero bucket size, this is not just a benchmarking omission on paper. The base `on_initialize` charge is the reservation the block builder uses for mandatory scheduler work before admitting ordinary extrinsics. If the timestamp housekeeping path is omitted from that reservation, each normal block begins from a slightly overstated remaining budget.

The result is live underpricing of per-block scheduler work. On blocks that exercise timestamp agendas, user transactions can be admitted against a budget that did not fully account for the mandatory housekeeping the runtime was always going to perform anyway.

Recommendation
Benchmark the full `on_initialize` base path as it runs in this fork, or add a dedicated timestamp base benchmark and charge both components explicitly in `on_initialize`.

EQ-QNT-SCHEDULER-F-08: v3 scheduler compatibility shims can panic on periodic requests

The local v3 compatibility shims abort with `assert!` if a caller asks for periodic scheduling. That path is currently latent in the checked-in runtime because the in-tree consumer passes `None` for `maybe_periodic`, but the shims sit on a production API surface and should fail closed with a normal dispatch error rather than panic.

Both compatibility shims reject periodic scheduling with `assert!`:

pallets/scheduler/src/lib.rs:1216-1234

```
1216 | // Shims for Substrate's v3 scheduler traits, required by pallet_referenda (external crate).
1217 | // Periodic scheduling is intentionally unsupported -- these impls exist solely to satisfy the
1218 | // trait bounds. Once pallet_referenda is forked locally, these can be removed entirely.
1219 | impl<T: Config> schedule::v3::Anon<BlockNumberFor<T>, <T as Config>::RuntimeCall,
1220 | T::PalletsOrigin>
1221 |   for Pallet<T>
1222 | {
1223 |   type Address = TaskAddressOf<T>;
1224 |   type Hasher = T::Hashing;
1225 |
1226 |   fn schedule(
```

```
1227 |     when: DispatchBlock<BlockNumberFor<T>>,
1228 |     maybe_periodic: Option<schedule::Period<BlockNumberFor<T>>>,
1229 |     priority: schedule::Priority,
1230 |     origin: T::PalletsOrigin,
1231 |     call: BoundedCallOf<T>,
1232 |   ) -> Result<Self::Address, DispatchError> {
1233 |     assert!(maybe_periodic.is_none(), "Periodic scheduling is not supported");
1234 |     Self::do_schedule(when.into(), priority, origin, call)
        }
```

pallets/scheduler/src/lib.rs:1258-1277

```
1258 | impl<T: Config> schedule::v3::Named<BlockNumberFor<T>, <T as Config>::RuntimeCall,
1259 | T::PalletsOrigin>
1260 |   for Pallet<T>
1261 | {
1262 |   type Address = TaskAddressOf<T>;
1263 |   type Hasher = T::Hashing;
1264 |
1265 |   fn schedule_named(
1266 |     id: TaskName,
1267 |     when: DispatchBlock<BlockNumberFor<T>>,
1268 |     maybe_periodic: Option<schedule::Period<BlockNumberFor<T>>>,
1269 |     priority: schedule::Priority,
1270 |     origin: T::PalletsOrigin,
1271 |     call: BoundedCallOf<T>,
1272 |   ) -> Result<Self::Address, DispatchError> {
1273 |     assert!(
1274 |       maybe_periodic.is_none(),
1275 |       "Periodic scheduling is not supported for schedule named"
1276 |     );
1277 |     Self::do_schedule_named(id, when.into(), priority, origin, call)
        }
```

The runtime wires these shims into the active scheduler implementation:

runtime/src/configs/mod.rs:394-406

```
394 | impl pallet_scheduler::Config for Runtime {
395 |   type RuntimeOrigin = RuntimeOrigin;
396 |   type PalletsOrigin = OriginCaller;
397 |   type RuntimeCall = RuntimeCall;
398 |   type MaximumWeight = MaximumSchedulerWeight;
399 |   type ScheduleOrigin = EnsureRoot<AccountId>;
400 |   type MaxScheduledPerBlock = MaxScheduledPerBlock;
401 |   type WeightInfo = pallet_scheduler::weights::SubstrateWeight<Runtime>;
402 |   type OriginPrivilegeCmp = frame_support::traits::EqualPrivilegeOnly;
403 |   type Preimages = Preimage;
404 |   type TimeProvider = Timestamp;
405 |   type Moment = u64;
406 |   type TimestampBucketSize = TimestampBucketSize;
```

If a future upstream referenda change, local pallet integration, or maintenance mistake starts passing `Some(period)`, block execution will panic instead of returning a recoverable scheduling error. That makes this a compatibility and robustness issue even though the current shipped call sites do not exercise it today.

Recommendation
Replace both `assert!` calls with an ordinary error return such as `DispatchError::Other` or a pallet-specific unsupported-mode error.

EQ-QNT-SCHEDULER-F-01: Timestamp-based `DispatchTime::After` is bucketed and behaves like absolute time

The custom timestamp-based scheduling path exposes the same `DispatchTime::After` surface as the block-number scheduler, but it does not provide the same contract. Block-number `After` is relative to the current block. Timestamp `After`, by contrast, is immediately normalised into a bucketed target time and then compared as an absolute timestamp.

The bucket normalization helper silently collapses division failure into zero:

primitives/scheduler/src/lib.rs:48-59

```
48 |   /// Normalize timestamp value
49 |   pub fn normalize(&self, precision: Moment) -> Self {
50 |     match self {
51 |       BlockNumberOrTimestamp::BlockNumber(_) => *self,
52 |       BlockNumberOrTimestamp::Timestamp(t) => {
53 |         let stripped_t =
54 |           t.checked_div(&precision).unwrap_or(Zero::zero()).saturating_mul(precision);
55 |
56 |         BlockNumberOrTimestamp::Timestamp(stripped_t.saturating_add(precision))
57 |       },
58 |     }
59 |   }
```

And the timestamp-based `DispatchTime::After` path immediately routes through that normalization:

pallets/scheduler/src/lib.rs:591-612

```
591 |   fn resolve_time(
592 |     when: DispatchTime<BlockNumberFor<T>, T::Moment>,
593 |   ) -> Result<BlockNumberOrTimestampOf<T>, DispatchError> {
594 |     let current_block = frame_system::Pallet::<T>::block_number();
595 |     let now = T::TimeProvider::now();
596 |
597 |     let when = match when {
598 |       DispatchTime::At(x) => BlockNumberOrTimestamp::BlockNumber(x),
599 |       // The current block has already completed it's scheduled tasks, so
```

```
600 |       // Schedule the task at lest one block after this current block.
601 |       DispatchTime::After(x) => {
602 |         // get the median block time
603 |         let res = match x {
604 |           BlockNumberOrTimestamp::BlockNumber(x) => BlockNumberOrTimestamp::BlockNumber(
605 |             current_block.saturating_add(x).saturating_add(One::one()),
606 |           ),
607 |           BlockNumberOrTimestamp::Timestamp(_) =>
608 |             x.normalize(T::TimestampBucketSize::get()),
609 |         };
610 |         res
611 |       },
612 |     };
```

The `reversible-transfers` pallet already works around this semantic mismatch by adding the current time before constructing `DispatchTime::After(Timestamp(...))`:

pallets/reversible-transfers/src/lib.rs:711-719

```
711 |       let dispatch_time = match delay {
712 |         BlockNumberOrTimestamp::BlockNumber(blocks) => DispatchTime::At(
713 |           <T as pallet::Config>::BlockNumberProvider::current_block_number()
714 |             .saturating_add(blocks),
715 |         ),
716 |         BlockNumberOrTimestamp::Timestamp(millis) =>
717 |           DispatchTime::After(BlockNumberOrTimestamp::Timestamp(
718 |             T::TimeProvider::now().saturating_add(millis),
719 |           )),
```

The current bucket size is derived from a 12-second target block time:

runtime/src/lib.rs:89-90

```
 89 | // Time is measured by number of blocks.
 90 | pub const TARGET_BLOCK_TIME_MS: u64 = 12_000;
```

runtime/src/configs/mod.rs:141-144

```
141 | parameter_types! {
142 |   /// Target block time ms
143 |   pub const TargetBlockTime: u64 = TARGET_BLOCK_TIME_MS;
144 |   pub const TimestampBucketSize: u64 = 2 * TARGET_BLOCK_TIME_MS; // Nyquist frequency
```

This is already more than an internal implementation quirk. `reversible-transfers` now compensates for the current behaviour locally, which means the bucketed interpretation has effectively become part of the runtime contract for downstream callers.

This has several important consequences:

- a zero `precision` value in `normalize` silently collapses through a checked division failure,
- `DispatchTime::After(Timestamp(x))` is semantically closer to "schedule at or around timestamp x" than to a relative "after x time units" API,
- `reversible-transfers` already compensates for this by converting a relative delay into an absolute timestamp before calling the scheduler,
- `normalize` always advances timestamps to the next bucket boundary, including exact boundaries, which makes the bucket semantics more aggressive than the API name suggests,
- bucket normalisation means a task can be executed at the start of the matching bucket rather than at or after the exact requested timestamp,
- with the current 24-second bucket size, execution can happen meaningfully earlier than a caller might expect before block-time variance is even considered.

The current API therefore communicates more symmetry and precision than the code actually provides. It also creates coupling for downstream pallets: `reversible-transfers` already compensates for the current behaviour locally, so any future semantic correction needs to be coordinated with callers that now depend on the existing bucketed interpretation.

Recommendation
Either preserve the exact target timestamp through scheduling and comparison, or rename and document the API so callers understand that timestamp dispatch is bucketed and not a precise after-time. Any change in semantics should be deployed together with downstream callers that currently compensate for the existing behaviour.

An explicit API shape such as the following would communicate the distinction directly:

```
enum DispatchTime {
    FromBlock(BlockNumber),
    AfterBlocks(BlockNumber),
    FromTime(Moment),
    AfterTime(Moment),
}
```

EQ-QNT-SCHEDULER-F-02: Split agenda execution counters can misclassify tasks as permanently overweight

The scheduler maintains separate local `executed` counters for block-based and timestamp-based agendas. Each counter is then used to determine whether the current task is the first task in its domain and therefore should be treated as permanently overweight if it does not fit.

The block and timestamp service loops each initialize their own local `executed` counter and pass it into `service_agenda`:

pallets/scheduler/src/lib.rs:839-860

```
839 |   fn service_block_agendas(weight: &mut WeightMeter, current_block: BlockNumberFor<T>, max: u32)
840 | {
841 |     let next_block = current_block.saturating_add(One::one());
842 |     let start_block = IncompleteBlockSince::<T>::take().unwrap_or(current_block);
843 |     let mut when = start_block;
844 |     let mut incomplete_since = next_block;
    |     let mut executed = 0;
    | ---------------------------------------------------------------------------------------
850 |     while count_down > 0 &&
851 |       when <= current_block &&
852 |       weight.can_consume(service_agenda_base_weight)
853 |     {
854 |       if !Self::service_agenda(
855 |         weight,
856 |         &mut executed,
857 |         BlockNumberOrTimestamp::BlockNumber(current_block),
858 |         BlockNumberOrTimestamp::BlockNumber(when),
859 |         u32::max_value(),
860 |       ) {
```

pallets/scheduler/src/lib.rs:876-915

```
876 |   fn service_timestamp_agendas(weight: &mut WeightMeter, current_time: T::Moment, max: u32) {
    | ---------------------------------------------------------------------------------------
896 |     let mut when = start_time;
897 |     let mut incomplete_since = next_bucket;
898 |     let mut executed = 0;
899 |
900 |     let max_items = T::MaxScheduledPerBlock::get();
901 |     let mut count_down = max;
902 |     let service_agenda_base_weight = T::WeightInfo::service_agenda_base(max_items);
903 |
904 |     while count_down > 0 &&
905 |       when <= normalized_time &&
906 |       weight.can_consume(service_agenda_base_weight)
907 |     {
908 |       log::debug!(target: "scheduler", "service_timestamp_agendas: when: {:?}", when);
909 |       if !Self::service_agenda(
910 |         weight,
911 |         &mut executed,
912 |         BlockNumberOrTimestamp::Timestamp(normalized_time),
913 |         BlockNumberOrTimestamp::Timestamp(when),
914 |         u32::MAX,
915 |       ) {
```

That counter then feeds the permanent-overweight decision:

pallets/scheduler/src/lib.rs:993-994

```
993 |       // Execute the task.
994 |       let result = Self::service_task(weight, now, when, agenda_index, *executed == 0, task);
```

pallets/scheduler/src/lib.rs:1077-1086

```
1077 |     match Self::execute_dispatch(weight, task.origin.clone(), call) {
1078 |       Err(()) if is_first => {
1079 |         T::Preimages::drop(&task.call);
1080 |         Self::deposit_event(Event::PermanentlyOverweight {
1081 |           task: (when, agenda_index),
1082 |           id: task.maybe_id,
1083 |         });
1084 |         Err((Unavailable, None))
1085 |       },
1086 |       Err(()) => Err((Overweight, Some(task))),
```

In the custom split-agenda design this is no longer a safe assumption. Timestamp agendas execute in the same block budget as block agendas, but the first timestamp task does not see the work that has already been consumed by earlier block tasks. This means a large timestamp task can be removed as permanently overweight even though it would fit normally in a later block when considered against a fresh budget.

Downstream impact on reversible-transfers

The most concrete downstream impact is in `pallet-reversible-transfers`, which places funds on hold before scheduling a named `execute_transfer` task. The failure chain is:

1. `schedule_transfer` stores pending-transfer metadata, places funds on hold, and schedules `execute_transfer` as a named task.
2. Scheduler servicing removes the named-task lookup before the task outcome is known.
3. If earlier block-agenda work has already consumed enough budget, the first timestamp task can be misclassified as permanently overweight because it sees a fresh per-domain `executed` counter.
4. The scheduler then treats the task as terminal, drops the preimage reference, and clears the agenda slot.
5. A later `cancel_transfer` call asks the scheduler to `cancel_named`, but that lookup has already been removed, so cancellation fails with `CancellationFailed`.
6. The user is left looking at held funds and pending-transfer state while ordinary cancellation keeps failing. High-security accounts can still use interceptor-driven `recover_funds`, but ordinary accounts cannot clear the state themselves.

So the split-counter bug can leave an ordinary delayed transfer in a bad terminal state: the hold remains, the pending transfer remains, but the named scheduler task is gone and ordinary cancellation no longer works.

Note that retries would not fix this path. Once the task is misclassified as permanently overweight, the scheduler treats it as terminal rather than retryable.

Recommendation
Share the executed counter across both agenda domains so the “first task” logic matches real block execution rather than per-domain bookkeeping.

EQ-QNT-SCHEDULER-F-03: Retry configuration accepts mismatched block and timestamp period types

The retry configuration stores its retry period as `BlockNumberOrTimestamp`, which means a caller can attach a block-number retry period to a timestamp-scheduled task or vice versa. A test already exists for the resulting failure mode, so this is a known issue rather than an unknown edge case.

The retry setter accepts either period type without checking it against the task address:

pallets/scheduler/src/lib.rs:501-519

```
501 |     pub fn set_retry(
502 |       origin: OriginFor<T>,
503 |       task: TaskAddressOf<T>,
504 |       retries: u8,
505 |       period: BlockNumberOrTimestampOf<T>,
506 |     ) -> DispatchResult {
507 |       T::ScheduleOrigin::ensure_origin(origin.clone())?;
508 |       let origin = <T as Config>::RuntimeOrigin::from(origin);
509 |       let (when, index) = task;
510 |       let agenda = Agenda::<T>::get(when);
511 |       let scheduled = agenda
512 |         .get(index as usize)
513 |         .and_then(Option::as_ref)
514 |         .ok_or(Error::<T>::NotFound)?;
515 |       Self::ensure_privilege(origin.caller(), &scheduled.origin)?;
516 |       Retries::<T>::insert(
517 |         (when, index),
518 |         RetryConfig { total_retries: retries, remaining: retries, period },
519 |       );
```

The mismatch is only discovered later if retry scheduling fails:

pallets/scheduler/src/lib.rs:1177-1195

```
1177 |     match now.saturating_add(&period) {
1178 |       Ok(wake) => match Self::place_task(wake, task.as_retry()) {
1179 |         Ok(address) => {
1180 |           Retries::<T>::insert(address, RetryConfig { total_retries, remaining, period });
1181 |         },
1182 |         Err((_, task)) => {
1183 |           T::Preimages::drop(&task.call);
1184 |           Self::deposit_event(Event::RetryFailed {
1185 |             task: (when, agenda_index),
1186 |             id: task.maybe_id,
1187 |           });
```

```
1188 |         },
1189 |       },
1190 |       Err(_) => {
1191 |         Self::deposit_event(Event::RetryFailed {
1192 |           task: (when, agenda_index),
1193 |           id: task.maybe_id,
1194 |         });
1195 |       },
```

The problem is not deep: the pallet already has access to the scheduled task address when the retry configuration is written. It therefore has enough information to reject a mismatched retry type at the point where the configuration is stored, instead of discovering the mismatch only later when a task fails and retry scheduling is attempted.

Recommendation
Validate that retry period type matches task scheduling type before inserting retry configuration into storage.

EQ-QNT-SCHEDULER-F-04: Retry configuration is lost when a task is rescheduled

Retry configuration is stored by task address. Both `do_reschedule` and `do_reschedule_named` move the task to a new address, but neither function moves the associated `Retries` entry. The retry policy is therefore left behind at the old address and silently stops applying to the rescheduled task.

Retry configuration is written against the current task address for ordinary tasks:

pallets/scheduler/src/lib.rs:501-519

```
501 |     pub fn set_retry(
502 |       origin: OriginFor<T>,
503 |       task: TaskAddressOf<T>,
504 |       retries: u8,
505 |       period: BlockNumberOrTimestampOf<T>,
506 |     ) -> DispatchResult {
507 |       T::ScheduleOrigin::ensure_origin(origin.clone())?;
508 |       let origin = <T as Config>::RuntimeOrigin::from(origin);
509 |       let (when, index) = task;
    ...
516 |       Retries::<T>::insert(
517 |         (when, index),
518 |         RetryConfig { total_retries: retries, remaining: retries, period },
519 |       );
```

Named tasks follow the same address-keyed pattern:

pallets/scheduler/src/lib.rs:537-555

```
537 |     pub fn set_retry_named(
538 |       origin: OriginFor<T>,
539 |       id: TaskName,
540 |       retries: u8,
541 |       period: BlockNumberOrTimestampOf<T>,
542 |     ) -> DispatchResult {
543 |       T::ScheduleOrigin::ensure_origin(origin.clone())?;
544 |       let origin = <T as Config>::RuntimeOrigin::from(origin);
545 |       let (when, agenda_index) = Lookup::<T>::get(&id).ok_or(Error::<T>::NotFound)?;
---
552 |       Retries::<T>::insert(
553 |         (when, agenda_index),
554 |         RetryConfig { total_retries: retries, remaining: retries, period },
555 |       );
```

Cancellation explicitly removes `Retries` entries at the old address:

pallets/scheduler/src/lib.rs:710-777

```
710 |     if let Some(s) = scheduled {
711 |       T::Preimages::drop(&s.call);
712 |       if let Some(id) = s.maybe_id {
713 |         Lookup::<T>::remove(id);
714 |       }
715 |       Retries::<T>::remove((when, index));
716 |       Self::cleanup_agenda(when);
---
775 |             if let Some(ref s) = s {
776 |               Retries::<T>::remove((when, index));
777 |               T::Preimages::drop(&s.call);
```

The unnamed reschedule path moves the task without moving the retry configuration:

pallets/scheduler/src/lib.rs:724-742

```
724 |   fn do_reschedule(
725 |     (when, index): TaskAddressOf<T>,
726 |     new_time: DispatchTime<BlockNumberFor<T>, T::Moment>,
727 |   ) -> Result<TaskAddressOf<T>, DispatchError> {
---
734 |     let task = Agenda::<T>::try_mutate(when, |agenda| {
735 |       let task = agenda.get_mut(index as usize).ok_or(Error::<T>::NotFound)?;
736 |       ensure!(!matches!(task, Some(Scheduled { maybe_id: Some(_), .. })), Error::<T>::Named);
737 |       task.take().ok_or(Error::<T>::NotFound)
738 |     })?;
739 |     Self::cleanup_agenda(when);
740 |     Self::deposit_event(Event::Canceled { when, index });
741 |
742 |     Self::place_task(new_time, task).map_err(|x| x.0)
```

The named reschedule path does the same thing:

pallets/scheduler/src/lib.rs:792-811

```
792 |   fn do_reschedule_named(
793 |     id: TaskName,
794 |     new_time: DispatchTime<BlockNumberFor<T>, T::Moment>,
795 |   ) -> Result<TaskAddressOf<T>, DispatchError> {
---
798 |     let lookup = Lookup::<T>::get(id);
799 |     let (when, index) = lookup.ok_or(Error::<T>::NotFound)?;
---
805 |     let task = Agenda::<T>::try_mutate(when, |agenda| {
806 |       let task = agenda.get_mut(index as usize).ok_or(Error::<T>::NotFound)?;
807 |       task.take().ok_or(Error::<T>::NotFound)
808 |     })?;
809 |     Self::cleanup_agenda(when);
810 |     Self::deposit_event(Event::Canceled { when, index });
811 |     Self::place_task(new_time, task).map_err(|x| x.0)
```

Later, `service_task` looks up retries at the task's current address:

pallets/scheduler/src/lib.rs:1088-1089

```
1088 |         let failed = result.is_err();
1089 |         let maybe_retry_config = Retries::<T>::take((when, agenda_index));
```

If the task has been rescheduled, that lookup finds nothing. A task that would otherwise have been retried therefore runs once, fails, and loses its retry policy entirely.

The abandoned `Retries` entry is also left behind at the old address. Repeated operator-driven reschedules can therefore accumulate orphaned retry records even though only the moved task remains live.

Recommendation
When a task is rescheduled, move any existing `Retries` entry from the old address to the new one after the task has been placed successfully.

EQ-QNT-SCHEDULER-F-06: Retry cloning does not preserve preimage ownership for lookup-backed tasks

When a failed task is retried, the scheduler clones the scheduled call into a new future slot without acquiring a matching preimage reference for that clone. The current public scheduling extrinsics note the preimage before inserting the task, so this is not a universal retry break. The retry path itself, however, still performs unbalanced preimage accounting for lookup-backed calls.

Ordinary schedule paths request preimage ownership after storing the task:

pallets/scheduler/src/lib.rs:679-762

```
679 |   fn do_schedule(
680 |     when: DispatchTime<BlockNumberFor<T>, T::Moment>,
681 |     priority: schedule::Priority,
682 |     origin: T::PalletsOrigin,
683 |     call: BoundedCallOf<T>,
684 |   ) -> Result<TaskAddressOf<T>, DispatchError> {
685 |     let when = Self::resolve_time(when)?;
686 |     let lookup_hash = call.lookup_hash();
687 |     let task = Scheduled { maybe_id: None, priority, call, origin, _phantom: PhantomData };
688 |     let res = Self::place_task(when, task).map_err(|x| x.0)?;
689 |     if let Some(hash) = lookup_hash {
690 |       T::Preimages::request(&hash);
691 |     }
```

```
745 |   fn do_schedule_named(
746 |     id: TaskName,
747 |     when: DispatchTime<BlockNumberFor<T>, T::Moment>,
748 |     priority: schedule::Priority,
749 |     origin: T::PalletsOrigin,
750 |     call: BoundedCallOf<T>,
751 |   ) -> Result<TaskAddressOf<T>, DispatchError> {
752 |     if Lookup::<T>::contains_key(&id) {
753 |       return Err(Error::<T>::FailedToSchedule.into());
754 |     }
755 |     let when = Self::resolve_time(when)?;
756 |     let lookup_hash = call.lookup_hash();
757 |     let task =
758 |       Scheduled { maybe_id: Some(id), priority, call, origin, _phantom: Default::default() };
759 |     let res = Self::place_task(when, task).map_err(|x| x.0)?;
760 |     if let Some(hash) = lookup_hash {
761 |       T::Preimages::request(&hash);
762 |     }
```

The retry path clones the task with `as_retry()` and inserts it without a matching `T::Preimages::request(...)` call:

pallets/scheduler/src/lib.rs:148-1184

```
148 |   pub fn as_retry(&self) -> Self {
149 |     Self {
150 |       maybe_id: None,
151 |       priority: self.priority,
152 |       call: self.call.clone(),
153 |       origin: self.origin.clone(),
154 |       _phantom: Default::default(),
```

```
1153 |   fn schedule_retry(
1154 |     weight: &mut WeightMeter,
1155 |     now: BlockNumberOrTimestampOf<T>,
1156 |     when: BlockNumberOrTimestampOf<T>,
1157 |     agenda_index: u32,
1158 |     task: &ScheduledOf<T>,
1159 |     retry_config: RetryConfig<BlockNumberOrTimestampOf<T>>,
1160 |   ) {
1161 |     if weight
1162 |       .try_consume(T::WeightInfo::schedule_retry(T::MaxScheduledPerBlock::get()))
1163 |       .is_err()
1164 |     {
1165 |       Self::deposit_event(Event::RetryFailed {
1166 |         task: (when, agenda_index),
1167 |         id: task.maybe_id,
1168 |       });
1169 |       return;
1170 |     }
1171 |
1172 |     let RetryConfig { total_retries, mut remaining, period } = retry_config;
1173 |     remaining = match remaining.checked_sub(1) {
1174 |       Some(n) => n,
1175 |       None => return,
1176 |     };
1177 |     match now.saturating_add(&period) {
1178 |       Ok(wake) => match Self::place_task(wake, task.as_retry()) {
1179 |         Ok(address) => {
1180 |           Retries::<T>::insert(address, RetryConfig { total_retries, remaining, period });
1181 |         },
1182 |         Err((_, task)) => {
1183 |           T::Preimages::drop(&task.call);
1184 |           Self::deposit_event(Event::RetryFailed {
```

After retry scheduling, `service_task` still drops the original task's preimage reference unconditionally:

pallets/scheduler/src/lib.rs:1097-1103

```
1097 |         if let Some(retry_config) = maybe_retry_config {
1098 |           if failed {
1099 |             Self::schedule_retry(weight, now, when, agenda_index, &task, retry_config);
1100 |           }
1101 |         }
```

```
1102 |
1103 |         T::Preimages::drop(&task.call);
```

This means the retry clone is not carrying explicit ownership of the lookup-backed call it now depends on. The `Err` branch in `schedule_retry` is also unbalanced in the opposite direction: it drops the cloned task even though that clone never acquired its own request in the first place.

Recommendation
Handle retry preimage ownership explicitly. Either request a new reference for the retry clone after it is placed successfully, or transfer the original task's ownership instead of dropping it unconditionally. In either model, remove the spurious `drop` on the failed retry-placement branch.

EQ-QNT-SCHEDULER-O-01: Scheduler fork and refactor cleanup remains unfinished

Several comments, metadata entries, linting choices, and duplicated interfaces still look like fork-time shortcuts and refactoring leftovers.

Examples
One benchmark still carries a `pov_mode` annotation whose generated proof-size component is not meaningfully enforced or priced by the current runtime (EQ-QNT-RUNTIME-O-02):

pallets/scheduler/src/benchmarking.rs:167-169

```
167 |   #[pov_mode = MaxEncodedLen {
168 |     Preimage::PreimageFor: Measured
169 |   }]
```

`resolve_time` also still has comments that no longer line up cleanly with the match arms they describe:

pallets/scheduler/src/lib.rs:599-611

```
599 |       // The current block has already completed it's scheduled tasks, so
600 |       // Schedule the task at lest one block after this current block.
601 |       DispatchTime::After(x) => {
602 |         // get the median block time
603 |         let res = match x {
604 |           BlockNumberOrTimestamp::BlockNumber(x) => BlockNumberOrTimestamp::BlockNumber(
605 |             current_block.saturating_add(x).saturating_add(One::one()),
606 |           ),
607 |           BlockNumberOrTimestamp::Timestamp(_) =>
608 |             x.normalize(T::TimestampBucketSize::get()),
609 |         };
```

```
610 |         res
611 |       },
```

The crate simultaneously suppresses all clippy warnings at the top level:

pallets/scheduler/src/lib.rs:1-1

```
  1 | #![allow(clippy::all)]
```

And it still defines a local `ScheduleNamed` trait even though shared primitives already provide the version that is actually used:

pallets/scheduler/src/traits.rs:11-16

```
 11 | /// A trait for scheduling tasks with a name, and with an approximate dispatch time.
 12 | pub trait ScheduleNamed<BlockNumber, Moment, Call, Origin> {
 13 |   /// Address type for the scheduled task.
 14 |   type Address: Codec + MaxEncodedLen + Clone + Eq + EncodeLike + core::fmt::Debug;
 15 |   /// The type of the hash function used for hashing.
 16 |   type Hasher: Hash;
```

primitives/scheduler/src/lib.rs:133-138

```
133 | /// A trait for scheduling tasks with a name, and with an approximate dispatch time.
134 | pub trait ScheduleNamed<BlockNumber, Moment, Call, Origin> {
135 |   /// Address type for the scheduled task.
136 |   type Address: Codec + MaxEncodedLen + Clone + Eq + EncodeLike + core::fmt::Debug;
137 |   /// The type of the hash function used for hashing.
138 |   type Hasher: Hash;
```

None of these items are severe, but they materially increase the amount of archaeology required to understand the pallet.

Recommendation
Resolve the drift outlined above so the pallet's comments, metadata, and local interfaces match the code they describe.

EQ-QNT-SCHEDULER-O-02: Timestamp agenda paths are not fully benchmarked or tested

When timestamp-based agendas were added, benchmark and test coverage did not expand proportionally. The pallet's timestamp scheduling paths remain under-benchmarked, and several edge cases are still missing regression tests.

Most concretely, the generated `service_agendas_base` benchmark still exercises only `service_block_agendas(..., 0)` and never the timestamp housekeeping path that `on_initialize` now runs on every non-genesis block.

The retained follow-up list includes large time-gap servicing, invalid named-task lookup state, `set_retry` edge cases, `schedule_after(Timestamp(0))`, and cancellation behaviour after retry rescheduling. Those cases have been preserved in Appendix A.

Recommendation
Extend benchmark and regression coverage for timestamp agenda paths directly.

6. REVERSIBLE TRANSFERS

EQ-QNT-REVERSIBLE-F-01: High-security configuration is effectively permanent once enabled

The pallet treats high-security enrollment as live authorization state across the runtime, but it does not provide any on-chain path to rotate the interceptor, change the delay, or remove the configuration again. Even the emergency `recover_funds` flow drains balances without clearing the account's high-security binding.

`set_high_security` is write-once and only appends to the interceptor index:

pallets/reversible-transfers/src/lib.rs:343-372

```
343 |     pub fn set_high_security(
344 |       origin: OriginFor<T>,
345 |       delay: BlockNumberOrTimestampOf<T>,
346 |       interceptor: T::AccountId,
347 |     ) -> DispatchResult {
348 |       let who = ensure_signed(origin)?;
349 |
350 |       ensure!(interceptor != who.clone(), Error::<T>::InterceptorCannotBeSelf);
351 |       ensure!(
352 |         !HighSecurityAccounts::<T>::contains_key(&who),
353 |         Error::<T>::AccountAlreadyHighSecurity
    ---
361 |       InterceptorIndex::<T>::try_mutate(interceptor.clone(), |accounts| {
362 |         if !accounts.contains(&who) {
363 |           accounts
364 |             .try_push(who.clone())
365 |             .map_err(|_| Error::<T>::TooManyInterceptorAccounts)
366 |         } else {
367 |           Ok(())
368 |         }
369 |       })?;
370 |
371 |       HighSecurityAccounts::<T>::insert(who.clone(), &high_security_account_data);
372 |       Self::deposit_event(Event::HighSecuritySet { who, interceptor, delay });
```

The emergency recovery path cancels transfers and drains free balance, but it does not remove the account from `HighSecurityAccounts` or clean up `InterceptorIndex`:

pallets/reversible-transfers/src/lib.rs:494-540

```
494 |     pub fn recover_funds(
495 |       origin: OriginFor<T>,
496 |       account: T::AccountId,
497 |     ) -> DispatchResultWithPostInfo {
```

```
498 |       let who = ensure_signed(origin)?;
499 |
500 |       let high_security_account_data = HighSecurityAccounts::<T>::get(&account)
501 |         .ok_or(Error::<T>::AccountNotHighSecurity)?;
502 |
503 |       ensure!(who == high_security_account_data.interceptor, Error::<T>::InvalidReverser);
```

```
507 |       for tx_id in PendingTransfersBySender::<T>::take(&account).iter() {
508 |         if let Some(pending) = PendingTransfers::<T>::take(tx_id) {
509 |           let schedule_id = Self::make_schedule_id(tx_id).ok();
510 |           if let Some(id) = schedule_id {
511 |             let _ = T::Scheduler::cancel_named(id);
512 |           }
```

```
530 |       let call: RuntimeCallOf<T> = pallet_balances::Call::<T>::transfer_all {
531 |         dest: T::Lookup::unlookup(who.clone()),
532 |         keep_alive: false,
533 |       }
534 |       .into();
535 |
536 |       call.dispatch(frame_system::RawOrigin::Signed(account.clone()).into())?;
537 |
538 |       Self::deposit_event(Event::FundsRecovered { account, guardian: who });
539 |
540 |       Ok(Some(<T as Config>::WeightInfo::recover_funds(num_cancelled as u32)).into())
```

Consequences of permanence

A bad interceptor choice or a compromised interceptor key becomes a long-lived runtime binding. Once a high-security account is enrolled, every future outgoing transfer for that account remains tied to the same interceptor and delay configuration unless governance intervenes or storage is modified out of band.

That has several concrete consequences. A compromised interceptor can keep blocking or front-running the account's delayed-transfer workflow indefinitely, while the account owner has no self-service way to rotate to a new interceptor or change the configured delay. Even without compromise, a user who simply wants to retire the setup later cannot do so normally. The built-in `recover_funds` path is an emergency drain, not lifecycle management: it releases balance pressure, but it still leaves the high-security binding and index state behind.

The permanence also affects storage. `InterceptorIndex` only accumulates membership; even an interceptor that no longer has any useful operational relationship with an account keeps its index entry because there is no path that removes the binding cleanly.

Recommendation

At minimum, add explicit on-chain lifecycle management: a way to rotate the interceptor, a way to exit high-security mode, and consistent cleanup of both `HighSecurityAccounts` and

`InterceptorIndex`. Separately, decide what authorization model that lifecycle should use. A fully self-service exit is the simplest repair, but the runtime may instead want interceptor co-signature or some other controlled rotation flow. If irreversibility is intentional, document it as an explicit governance dependency rather than leaving it as an implicit pallet default.

EQ-QNT-REVERSIBLE-F-02: Asset scheduling reuses a native-only benchmarked weight

Both asset scheduling extrinsics reuse `WeightInfo::schedule_transfer()`, but the benchmark behind that weight exercises only the native-balance hold path. The asset branch goes through `pallet_assets_holder`, so the current weight model is at best an approximation and likely underprices asset scheduling slightly.

The asset extrinsics use the same weight function as the native transfer extrinsics:

pallets/reversible-transfers/src/lib.rs:406-464

```
406 |     #[pallet::weight(<T as Config>::WeightInfo::schedule_transfer())]
407 |     pub fn schedule_transfer(
```

```
447 |     #[pallet::weight(<T as Config>::WeightInfo::schedule_transfer())]
448 |     pub fn schedule_asset_transfer(
```

```
463 |     #[pallet::weight(<T as Config>::WeightInfo::schedule_transfer())]
464 |     pub fn schedule_asset_transfer_with_delay(
```

The only scheduling benchmark exercises the native transfer path:

pallets/reversible-transfers/src/benchmarking.rs:104-123

```
104 |   #[benchmark]
105 |   fn schedule_transfer() -> Result<(), BenchmarkError> {
```

```
116 |     let call = make_transfer_call::<T>(recipient.clone(), transfer_amount)?;
117 |     let global_nonce = GlobalNonce::<T>::get();
118 |     let tx_id = T::Hashing::hash_of(&(caller.clone(), call, global_nonce).encode());
119 |
120 |     let recipient_lookup = <T as frame_system::Config>::Lookup::unlookup(recipient);
121 |     // Schedule the dispatch
122 |     #[extrinsic_call]
123 |     _(RawOrigin::Signed(caller.clone()), recipient_lookup, transfer_amount.into());
```

In the checked-in runtime the asset branch is live and is backed by `pallet_assets_holder`:

runtime/src/configs/mod.rs:509-537

```
509 | impl pallet_assets::Config for Runtime {
528 |   type Holder = pallet_assets_holder::Pallet<Runtime>;
532 | }
533 |
534 | impl pallet_assets_holder::Config for Runtime {
535 |   type RuntimeEvent = RuntimeEvent;
536 |   type RuntimeHoldReason = RuntimeHoldReason;
537 | }
```

The shared scheduling logic is still O(1), so this is a bounded weight-accounting issue rather than a correctness flaw. The problem is that the constant used for pricing asset scheduling is based on the wrong branch.

Recommendation
Benchmark the asset scheduling path directly and either give it its own weight function or use the heavier branch as the shared schedule weight.

EQ-QNT-REVERSIBLE-F-03: Orphaned scheduled holds have no dedicated repair path

Once a scheduled hold is detached from its `PendingTransfer` metadata, the runtime has no dedicated path to release it. That state is not part of the expected happy path, but the pallet still contains execution and recovery branches that consume pending-transfer metadata without making successful hold release a hard prerequisite. Once that happens, the held amount can remain frozen with no ordinary pallet path to reconstruct or release it.

`recover_funds` removes pending-transfer state first and then only best-effort releases held funds:

pallets/reversible-transfers/src/lib.rs:494-538

```
494 |     pub fn recover_funds(
495 |       origin: OriginFor<T>,
496 |       account: T::AccountId,
497 |     ) -> DispatchResultWithPostInfo {
507 |       for tx_id in PendingTransfersBySender::<T>::take(&account).iter() {
508 |         if let Some(pending) = PendingTransfers::<T>::take(tx_id) {
509 |           let schedule_id = Self::make_schedule_id(tx_id).ok();
510 |           if let Some(id) = schedule_id {
511 |             let _ = T::Scheduler::cancel_named(id);
512 |           }
513 |
514 |           if let Err(e) = Self::release_held_funds_with_fee(&pending, &who, true) {
```

```
528 |       }
529 |
530 |       let call: RuntimeCallOf<T> = pallet_balances::Call::<T>::transfer_all {
531 |         dest: T::Lookup::unlookup(who.clone()),
532 |         keep_alive: false,
533 |       }
534 |       .into();
535 |
536 |       call.dispatch(frame_system::RawOrigin::Signed(account.clone()).into())?;
537 |
538 |       Self::deposit_event(Event::FundsRecovered { account, guardian: who });
```

`release_held_funds_with_fee` silently returns `Ok(())` if it cannot realize the stored call:

pallets/reversible-transfers/src/lib.rs:840-906

```
840 |     fn release_held_funds_with_fee(
841 |       pending: &PendingTransferOf<T>,
842 |       recipient: &T::AccountId,
843 |       apply_fee: bool,
844 |     ) -> DispatchResult {
845 |       let (fee_amount, remaining_amount) = if apply_fee {
846 |         let volume_fee = T::VolumeFee::get();
847 |         let fee = volume_fee * pending.amount;
848 |         (fee, pending.amount.saturating_sub(fee))
849 |       } else {
850 |         (Zero::zero(), pending.amount)
851 |       };
852 |
853 |       if let Ok((call, _)) = T::Preimages::realize::<RuntimeCallOf<T>>(&pending.call) {
    ...
904 |       }
905 |
906 |       Ok(())
```

The asset execution path also discards release failure before deleting the pending-transfer record:

pallets/reversible-transfers/src/lib.rs:608-651

```
608 |     fn do_execute_transfer(tx_id: &T::Hash) -> DispatchResultWithPostInfo {
    ...
615 |       // Release held funds and determine asset_id for transfer proof recording
616 |       let asset_id: Option<AssetIdOf<T>> =
617 |         if let Ok(pallet_assets::Call::transfer_keep_alive { id, .. }) =
618 |           call.clone().try_into()
619 |         {
620 |           // Assets transfer: release the held asset amount
621 |           let reason = Self::asset_hold_reason();
```

```
622 |           let _ = <AssetsHolderOf<T> as AssetsHold<AccountIdOf<T>>>::release(
623 |             id.clone().into(),
624 |             &reason,
625 |             &pending.from,
626 |             pending.amount,
627 |             Precision::Exact,
628 |           );
629 |           Some(id.into())
```

```
643 |         };
644 |
645 |       // Remove transfer from storage
646 |       PendingTransfers::<T>::remove(tx_id);
647 |
648 |       // Remove from sender's pending list
649 |       PendingTransfersBySender::<T>::mutate(&pending.from, |list| {
650 |         list.retain(|id| id != tx_id);
651 |       });
```

That means the pallet's existing cancel and recovery flows are cleanup paths, not repair paths. If an earlier failure already left a hold orphaned, neither the user nor the interceptor has a dedicated extrinsic for releasing that residual amount.

Recommendation

Add a recovery-only extrinsic, ideally root- or interceptor-gated, that can release an orphaned hold without relying on the original pending-transfer preimage. Also emit a distinct event when hold release is skipped so operators can detect the condition on-chain.

EQ-QNT-REVERSIBLE-O-01: Naming and inline documentation drift make the high-security model harder to follow

The reversible-transfers pallet is reasonably understandable once the reader has the full codebase context, but its naming and inline documentation are not yet stable enough to act as a clean external contract on their own.

Examples

The event documentation already uses mixed terms for the recovery actor:

pallets/reversible-transfers/src/lib.rs:262-286

```
262 |   pub enum Event<T: Config> {
263 |     /// A user has enabled their high-security settings.
264 |     /// [who, interceptor, recoverer, delay]
265 |     HighSecuritySet {
266 |       who: T::AccountId,
267 |       interceptor: T::AccountId,
```

```
268 |       delay: BlockNumberOrTimestampOf<T>,
269 |     },
270 |     /// A transaction has been intercepted and scheduled for delayed execution.
271 |     /// [from, to, interceptor, amount, tx_id, execute_at_moment]
272 |     TransactionScheduled {
```

```
285 |     /// Funds were recovered from a high security account by its guardian.
286 |     FundsRecovered { account: T::AccountId, guardian: T::AccountId },
```

The same role is described as an `interceptor`, a `guardian`, and a `recoverer`. That may be understandable once the reader has the full pallet and runtime context, but it still makes it harder to tell whether those names are meant to be synonyms or distinct actors in the model.

And `GlobalNonce` is named like a cryptographic nonce even though it is just a monotonic identifier:

pallets/reversible-transfers/src/lib.rs:255-258

```
255 |   /// Global nonce for generating unique transaction IDs.
256 |   #[pallet::storage]
257 |   #[pallet::getter(fn global_nonce)]
258 |   pub type GlobalNonce<T: Config> = StorageValue<_, u64, ValueQuery>;
```

The pallet's own helper comments also blur the intended call flow. `do_schedule_transfer` is documented as if the transaction extension invokes it, but the extension only validates submitted calls; the `schedule_transfer` extrinsic is the component that actually calls the helper. Likewise, `execute_transfer` depends on the scheduler dispatching it as the signed pallet account, which is correct but not stated very clearly in the surrounding comments:

pallets/reversible-transfers/src/lib.rs:327-332

```
327 |   impl<T: Config> Pallet<T> {
328 |     /// Enable high-security for the calling account with a specified
329 |     /// reversibility delay.
330 |     ///
331 |     /// Recoverer and interceptor (aka guardian) could be the same account or
332 |     /// different accounts.
```

pallets/reversible-transfers/src/lib.rs:387-401

```
387 |     /// Called by the Scheduler to finalize the scheduled task/call
388 |     ///
389 |     /// - `tx_id`: The unique id of the transaction to finalize and dispatch.
390 |     #[pallet::call_index(2)]
391 |     #[pallet::weight(<T as Config>::WeightInfo::execute_transfer())]
392 |     #[allow(clippy::useless_conversion)]
```

```
393 |     pub fn execute_transfer(
394 |       origin: OriginFor<T>,
395 |       tx_id: T::Hash,
396 |     ) -> DispatchResultWithPostInfo {
397 |       let who = ensure_signed(origin)?;
398 |
399 |       ensure!(who == Self::account_id(), Error::<T>::InvalidSchedulerOrigin);
400 |
401 |       Self::do_execute_transfer(&tx_id)
```

pallets/reversible-transfers/src/lib.rs:793-805

```
793 |     /// Schedules a runtime call for delayed execution using the pre-configured delay.
794 |     /// This is intended to be called by the `TransactionExtension`, NOT directly by users.
795 |     pub fn do_schedule_transfer(
796 |       origin: T::RuntimeOrigin,
797 |       dest: <<T as frame_system::Config>::Lookup as StaticLookup>::Source,
798 |       amount: BalanceOf<T>,
799 |     ) -> DispatchResult {
800 |       let who = ensure_signed(origin)?;
801 |       let HighSecurityAccountData { delay, interceptor, .. } =
802 |         Self::high_security_accounts(&who).ok_or(Error::<T>::AccountNotHighSecurity)?;
803 |
804 |       Self::do_schedule_transfer_inner(who, dest, interceptor, amount, delay, None)
805 |     }
```

Taken together, the drift spans role vocabulary (`interceptor`, `guardian`, `recoverer` used interchangeably), identifier naming (`GlobalNonce` for a plain counter), and origin expectations that are only implicit in helper comments.

Recommendation

Replace remaining `guardian` and `recoverer` references with `interceptor`, rename `GlobalNonce` to for example `NextTransactionId`, and update the remaining helper comments so they describe the actual scheduling and execution flow.

EQ-QNT-REVERSIBLE-O-02: README terminology and trait-location docs lag behind the current high-security design

The pallet README still reflects an earlier version of the high-security model in a few places. The current pallet and runtime are clearer than the README, but readers who start from the README can still pick up outdated terminology and trait-location details.

Examples

The README still uses the older reversible-account terminology and `set_reversability` name:

pallets/reversible-transfers/README.md:34-42

Storages

- `ReversibleAccounts`: list of accounts that are `reversible` accounts. Accounts can call `ReversibleTransfers::set_reversability` extrinsic to join this set.
- `PendingTransfers`: stores current pending dispatches for the user. Maps `tx_id` to `(caller, pending_dispatch)`. We store the caller so that we can validate the user who's canceling the dispatch.
- `AccountPendingIndex`: stores the current count of pending transactions for the user so that they don't exceed `MaxPendingPerAccount`

Notes

- Transaction id is `((who, call).hash())` where `who` is the account that called the transaction and `call` is the call itself. This is used to identify the transaction in the scheduler and preimage. For identical transfers, there is a counter in `PendingTransfer` to differentiate between them.

It also describes `HighSecurityInspector` as if the trait still lived in this pallet rather than in shared primitives:

pallets/reversible-transfers/README.md:44-46

High-Security Integration
This pallet provides the **HighSecurityInspector** trait for integrating high-security features with other pallets (like `pallet-multisig`).

pallets/reversible-transfers/README.md:69-76

Implementation
This pallet provides:

- Trait definition (`pub trait HighSecurityInspector`)
- Helper functions for runtime implementation:
  - `is_high_security_account(who)` - checks `HighSecurityAccounts` storage
  - `get_guardian(who)` - retrieves guardian from storage
- Default no-op implementation: `impl HighSecurityInspector for ()`

This looks like documentation drift from earlier renames and refactors rather than a conceptual design problem. The issue is that the README no longer matches the code readers are expected to integrate with today.

Recommendation
Refresh the README around the current high-security terminology, calls, and trait location so it matches the pallet and runtime as implemented.

7. CLASSIFICATIONS

We use standard CVSS severity classifications:

- **Critical**
- **High**
- **Medium**
- **Low**
- **None (Information)**

Generally, we mark findings Critical if an exploit or bug may cause complete system compromise, extended outage, material fund loss, or equivalent protocol failure. These issues should be fixed with highest priority.

We mark findings as High if an issue can realistically produce serious security impact, material degradation of service, or significant exposure of sensitive behaviour under practical attacker assumptions.

Findings marked as Medium have a narrower impact or require more specific conditions, but still represent a meaningful correctness or security concern that should be prioritised.

Low findings generally capture limited-impact weaknesses, incorrect assumptions, or engineering choices that can contribute to future defects even if they are not immediately exploitable.

Informational findings contain no current vulnerability and instead record maintainability concerns, documentation drift, design tradeoffs, or observations that deserve follow-up.

8. CONCLUSION

We identified 44 findings and observations across the Quantus runtime, with the heaviest concentration in the multisig pallet and the custom scheduler fork.

We recommend prioritising three areas before treating the reviewed behaviour as stable:

0. **Correct the multisig fee model so dispatched `RuntimeCall` execution is properly charged.** EQ-QNT-MULTISIG-F-01

1. **Stabilise the custom scheduler fork's semantics and housekeeping paths before downstream integrations depend on them as a settled contract.** EQ-QNT-SCHEDULER-F-07 EQ-QNT-SCHEDULER-F-01

2. **Define a clearer lifecycle for high-security reversible-transfer accounts, including how that configuration can be rotated or retired safely over time.** EQ-QNT-REVERSIBLE-F-01

The remaining issues include additional medium-severity scheduler and reversible-transfer concerns, followed by lower-severity findings and informational observations concerning configuration safety, documentation drift, weight accounting, naming, and missing tests. They do not all require immediate code changes, but they do represent technical debt that will make later maintenance and future review harder if left unaddressed.

We appreciate Quantus' collaboration throughout the review and thank the team for the opportunity to assess these runtime components.

9. APPENDIX A: Candidate Test Cases and Follow-Up Checks

The following test cases are listed here so that the main findings chapters can stay concise.

Multisig

No.	Candidate test
1	`approve` on an already-approved proposal, including emitted events
2	`proposal_nonce` overflow or saturation behaviour
3	`execute` on a proposal that dispatches back into `multisig`
4	`approve_dissolve` with `threshold = 1` and immediate dissolution
5	`claim_deposits` when the caller has zero expired proposals
6	`approve_dissolve` duplicate approval handling
7	`create_multisig` does not burn fees if deposit reservation fails
8	`propose` does not burn fees if deposit reservation fails
9	`claim_deposits` for a single-signer multisig
10	`DepositsClaimed.total_returned` remains consistent across a `ProposalDeposit` change
11	`cancel` on an already-approved proposal

Scheduler

No.	Candidate test
1	`service_timestamp_agendas` with a large time gap, such as 30 days
2	`do_cancel_named` when `Lookup` points to an empty agenda slot
3	`set_retry` with `retries = 0`
4	`schedule_after` with `Timestamp(0)`
5	`cancel_named` after a named task has failed and been retried via `schedule_retry`
6	`do_reschedule` on a failing task that already has a non-empty `Retries` entry

10. APPENDIX B: Issue Resolution Status

This appendix records follow-up remediation status and associated fix references for the findings in this report.

A. Substrate Runtime

Issue	Resolved
EQ-QNT-RUNTIME-O-01 Info Expected and unexpected errors are not separated consistently	chain#546, chain#559
EQ-QNT-RUNTIME-O-02 Info Proof-size weight is carried through the system but neither enforced nor priced	Documented: chain#546
EQ-QNT-RUNTIME-O-03 Info Several custom pallets do not declare an explicit storage-version baseline	chain#546, chain#556

B. Treasury

Issue	Resolved
EQ-QNT-TREASURY-F-01 Low Treasury account configuration falls back to the all-zero account	chain#546
EQ-QNT-TREASURY-F-02 Low Missing `TreasuryPortion` storage collapses silently to a valid zero share	chain#546
EQ-QNT-TREASURY-F-03 Low Shared runtime test genesis does not initialize treasury pallet storage	chain#546
EQ-QNT-TREASURY-O-01 Info Changing the treasury account only redirects future reward flow	chain#546
EQ-QNT-TREASURY-O-02 Info Treasury reward portion is stored as a `u8` despite being used as a percentage type	chain#546
EQ-QNT-TREASURY-O-03 Info The standalone treasury pallet adds boilerplate without clear separation of responsibilities	Documented: chain#559

EQ-QNT-TREASURY-O-04 Treasury-side mint failures are silent on-chain	Info	chain#559

C. Multisig

Issue		Resolved
EQ-QNT-MULTISIG-F-01 Dispatched `RuntimeCall` execution is not charged in the multisig fee model	High	chain#546, chain#559
EQ-QNT-MULTISIG-F-02 Dynamic proposal fee formula floors too early	Low	chain#474, chain#546
EQ-QNT-MULTISIG-F-03 Several secondary multisig paths do not return accurate weight information	Low	chain#546
EQ-QNT-MULTISIG-F-04 `claim_deposits` weight and benchmark assumptions do not match the stated worst case	Low	chain#517
EQ-QNT-MULTISIG-F-05 `DepositsClaimed.total_returned` can drift from the amounts actually unreserved	Low	chain#546
EQ-QNT-MULTISIG-F-06 Dissolution can be prevented by sending trivial native balances to the multisig	Low	chain#546
EQ-QNT-MULTISIG-F-07 Dissolution ignores non-native assets and can strand them	Low	chain#546
EQ-QNT-MULTISIG-O-01 High-security validation and call decoding are deferred later than necessary	Info	chain#546
EQ-QNT-MULTISIG-O-02 Signer normalisation and duplicate checks are split across multiple layers	Info	chain#546
EQ-QNT-MULTISIG-O-03 The extrinsic call boundary is looser than the pallet's own limits require	Info	chain#546

EQ-QNT-MULTISIG-O-04 Info Proposal lifecycle semantics and naming are inconsistent	chain#546
EQ-QNT-MULTISIG-O-05 Info Proposal identifiers saturate instead of failing explicitly at exhaustion	chain#546
EQ-QNT-MULTISIG-O-06 Info Proposal bookkeeping and mutation paths are more complicated than they need to be	chain#546
EQ-QNT-MULTISIG-O-07 Info Dissolution and asset-handling semantics need clearer documentation	chain#546
EQ-QNT-MULTISIG-O-08 Info The current fee attribution model favors simplicity over deposit-backed cleanup	Documented: chain#546
EQ-QNT-MULTISIG-O-09 Info Several proposal lifecycle edge cases lack direct test coverage	chain#546
EQ-QNT-MULTISIG-O-10 Info Legacy balance traits are used where newer fungible interfaces would be a better fit	
EQ-QNT-MULTISIG-O-11 Info Threshold-one multisigs introduce a special-case proposal path	chain#546
EQ-QNT-MULTISIG-O-12 Info Per-signer proposal limits rely on an implicit non-zero signer invariant	chain#546

D. Scheduler

Issue	Resolved
EQ-QNT-SCHEDULER-F-01 Low Timestamp-based `DispatchTime::After` is bucketed and behaves like absolute time	chain#517, chain#557
EQ-QNT-SCHEDULER-F-02 Medium Split agenda execution counters can misclassify tasks as permanently overweight	chain#517
EQ-QNT-SCHEDULER-F-03 Low Retry configuration accepts mismatched block and timestamp period types	chain#557

EQ-QNT-SCHEDULER-F-04 Low Retry configuration is lost when a task is rescheduled	chain#517
EQ-QNT-SCHEDULER-F-05 Medium Inherited upstream scheduler behaviours remain problematic in this fork	chain#557
EQ-QNT-SCHEDULER-F-06 Low Retry cloning does not preserve preimage ownership for lookup-backed tasks	chain#557
EQ-QNT-SCHEDULER-F-07 Medium Timestamp agenda housekeeping is missing from the base `on_initialize` weight	chain#557
EQ-QNT-SCHEDULER-F-08 Medium v3 scheduler compatibility shims can panic on periodic requests	chain#557
EQ-QNT-SCHEDULER-O-01 Info Scheduler fork and refactor cleanup remains unfinished	chain#557
EQ-QNT-SCHEDULER-O-02 Info Timestamp agenda paths are not fully benchmarked or tested	chain#557

E. Reversible Transfers

Issue	Resolved
EQ-QNT-REVERSIBLE-F-01 Medium High-security configuration is effectively permanent once enabled	Accepted risk: chain#559
EQ-QNT-REVERSIBLE-F-02 Low Asset scheduling reuses a native-only benchmarked weight	chain#556
EQ-QNT-REVERSIBLE-F-03 Medium Orphaned scheduled holds have no dedicated repair path	chain#556 Accepted risk: chain#559
EQ-QNT-REVERSIBLE-O-01 Info Naming and inline documentation drift make the high-security model harder to follow	chain#559
EQ-QNT-REVERSIBLE-O-02 Info README terminology and trait-location docs lag behind the current high-security design	chain#556